

## Modeling the Flow of Materials, Advanced

### Model the Flow of Materials, Advanced

In this chapter we introduce you to some of the more advanced features of modeling the flow of materials in your simulation model.

You will learn:

- How to [customize the behavior of the objects](#) with controls and user-defined attributes.
- How to [program entrance controls and exit controls](#) that override the built-in, default transfer behavior of the material flow objects.
- How to [define sensors](#) on length-oriented objects that are triggered by the *MUs* moving on them.
- How to [define observers](#) that are triggered when the value of a watchable *attribute*, *read-only attribute*, or *method* changes.
- How to [create and work with a distance control](#) to prevent *Transporters* from colliding.

Compare [Model the Flow of Materials, Basics](#)

Back to [Modeling the Material Flow](#)

### Customizing the Behavior of Objects

#### Customize the Behavior of Objects

You can customize the behavior of most of the *Plant Simulation* objects to meet your modeling needs.

- You can program and assign *control methods* to an object. The object then reacts to certain user actions, such as inserting and deleting objects, etc.
- You can create *user-defined attributes* for an object, which it does not provide by default.

Compare [Create Entrance and Exit Controls](#)

Back to [Model the Flow of Materials, Advanced](#)

#### Defining Controls

##### Define Controls

You can assign a *control method* that makes the object take an action you want it to execute.

- Program the actions you want one or several objects to take in a *Method* object and insert it into a *Frame* in your simulation model or in a folder in the *Class Library*. This way you program actions that several objects can use.
- Create a control that only applies to the selected object. Then you will program the actions in a user-defined attribute of data type *method* of this object. The control thus becomes part of the object and carries it with it, when you insert the object into other *Frames*.

### Compare [Edit Controls \[material flow objects\]](#)

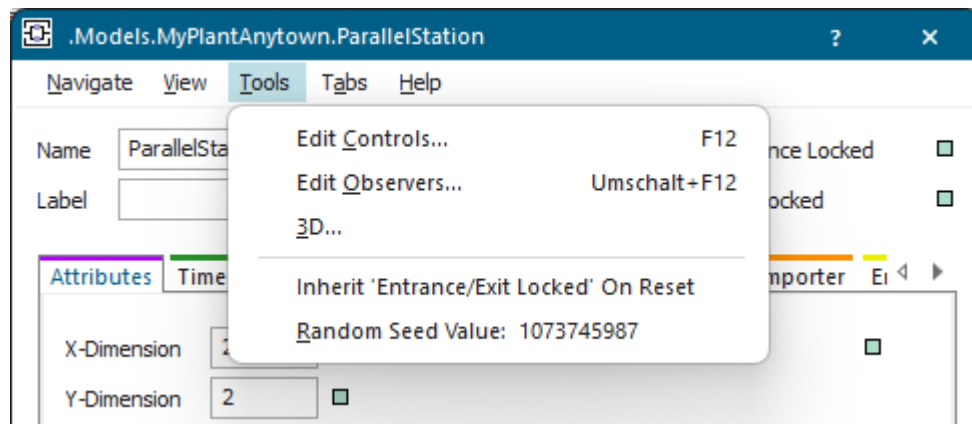
### Back to [Customize the Behavior of Objects](#)

## Assign a Control Method

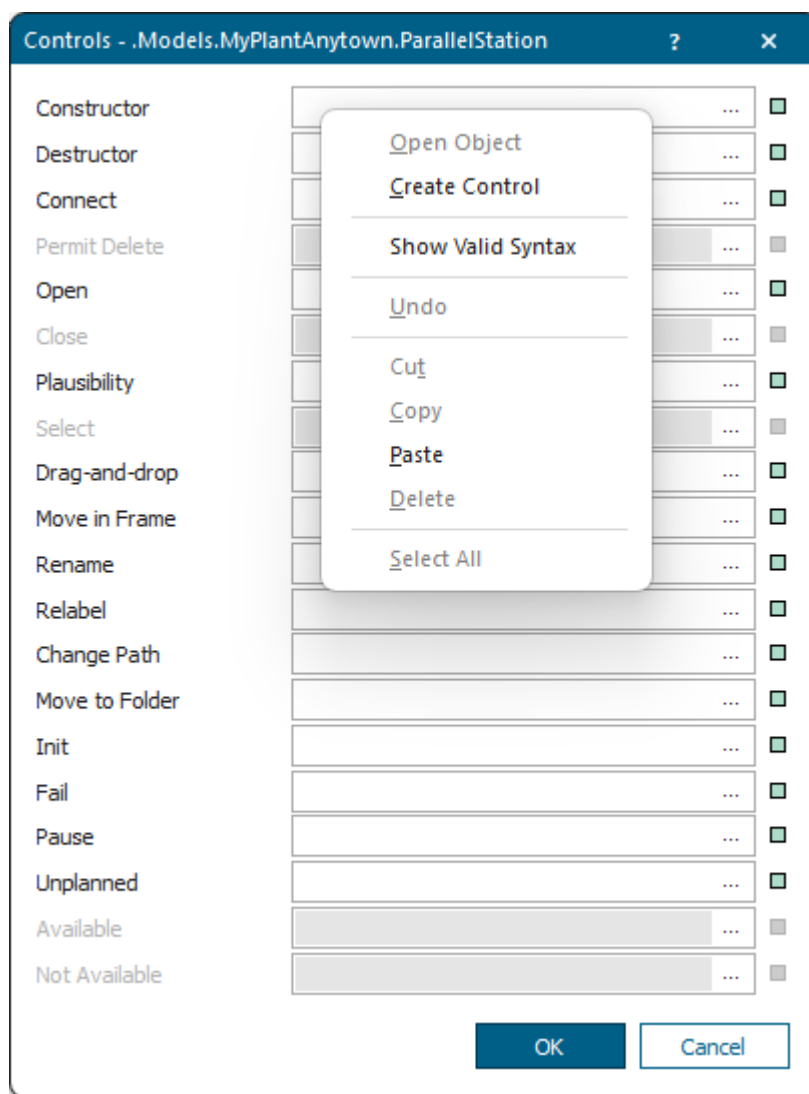
If you want a number of objects that are located in the same *Frame* to call the same *method*, you typically use a *Method* object, which you insert into that *Frame*.

To assign a control, which you programmed in a *Method* object:

- Program the control in a *Method* object.
- Click the object to which you want to assign the control in the *Frame*.
- Select **Tools > Edit Controls**.



- In the dialog **Controls** select the *Method* to be activated, when the user or when *Plant Simulation* performs a certain action:
  - Click  next to the name of the control. In the dialog [Select Object](#) navigate to the folder or *Frame*, in which the *Method* object is located, select it, and click **OK**.



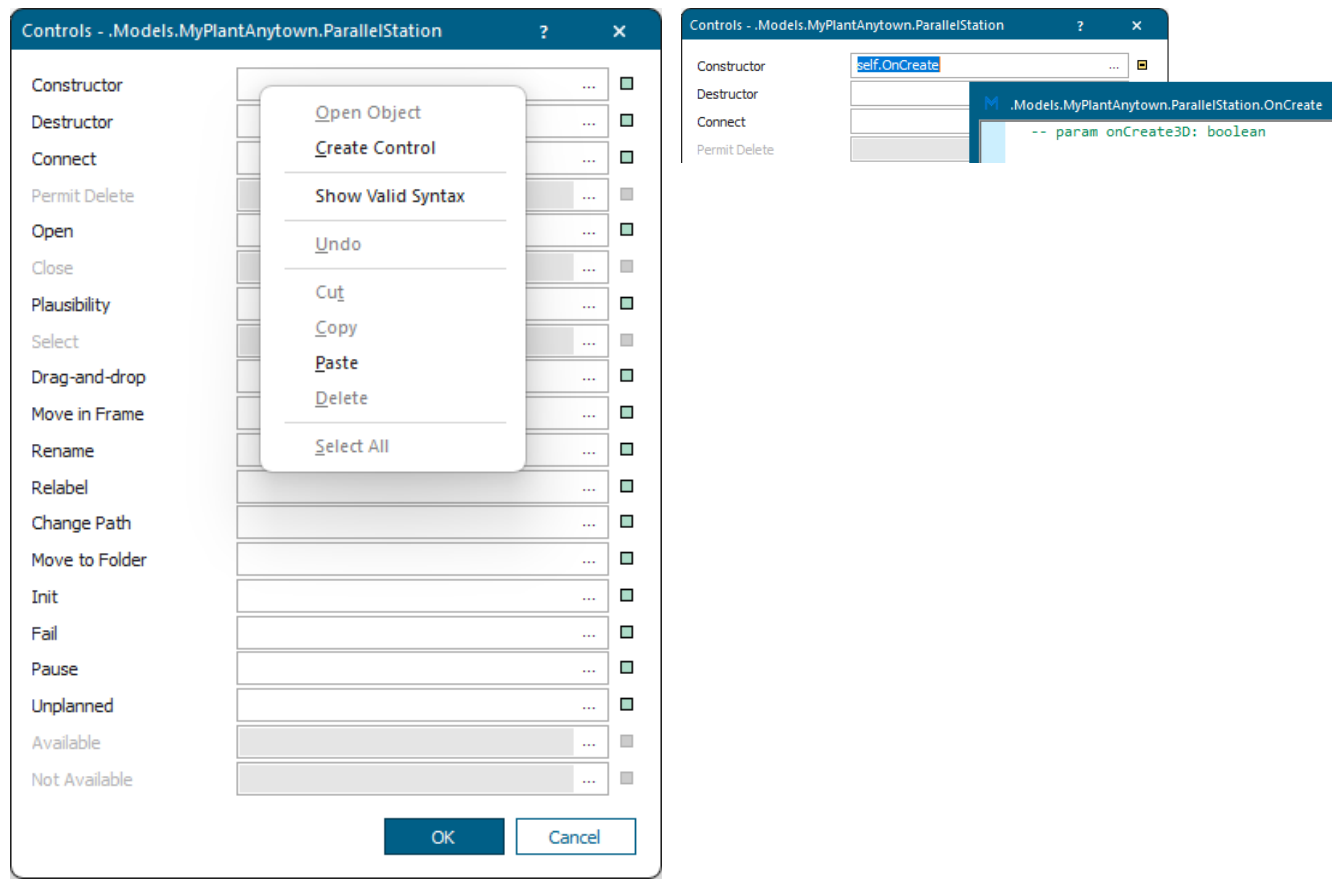
## Back to [Define Controls](#)

### Create a Control Which is Part of the Object

If you want all instances of a class to provide the same control, you typically create that control as a *user-defined attribute* in the class.

To create a control as a *user-defined attribute* of data type *method* of the object:

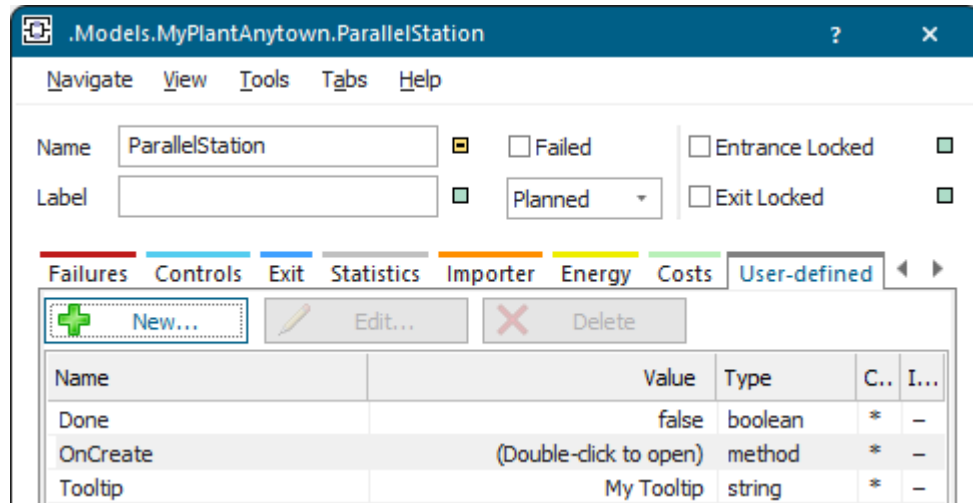
- Type a meaningful name into the text box, right-click in the text box and select **Create Control**. *Plant Simulation* then inserts `self.Name_you_typed_in_for_the_control`, such as `self.mySelectControl`.
- Right-click in the text box and select **Create Control** on the context menu. *Plant Simulation* then inserts `self.OnBuilt_in_name_of_the_control`, such as `self.OnCreate`.



Type the source code of this control into the *Method* that opens.

This control is a *user-defined attribute* of data type *method*. To open and modify it, do one of the following:

- Click in the text box and press **F2**.
- Hold down **Shift** and double-click in the text box.
- Select the tab **User-defined** and double-click the name of the *Method* in the list pane.



## Back to Define Controls

### Create a User-defined Attribute Manually

In addition to the built-in attributes, which are part of the *Plant Simulation* objects, you can add *user-defined attributes*, also called *attribute methods*, to meet your modeling needs to most objects. Naturally, these attributes do not affect the built-in properties of the object.

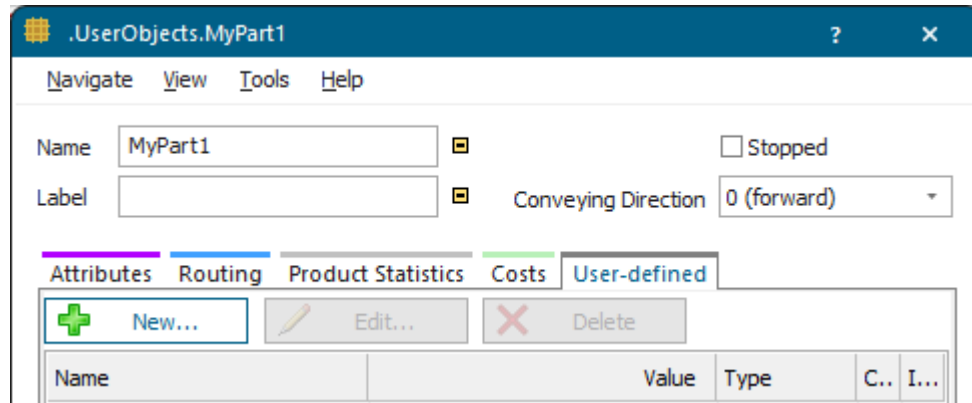
A *user-defined attribute* provides most of the functions that a global variable in any of the programming languages makes available. As a rule you will use a *user-defined attribute* for internal purposes, for example to attach the type of article, the order number, etc. to a *MU*.

You might, for example:

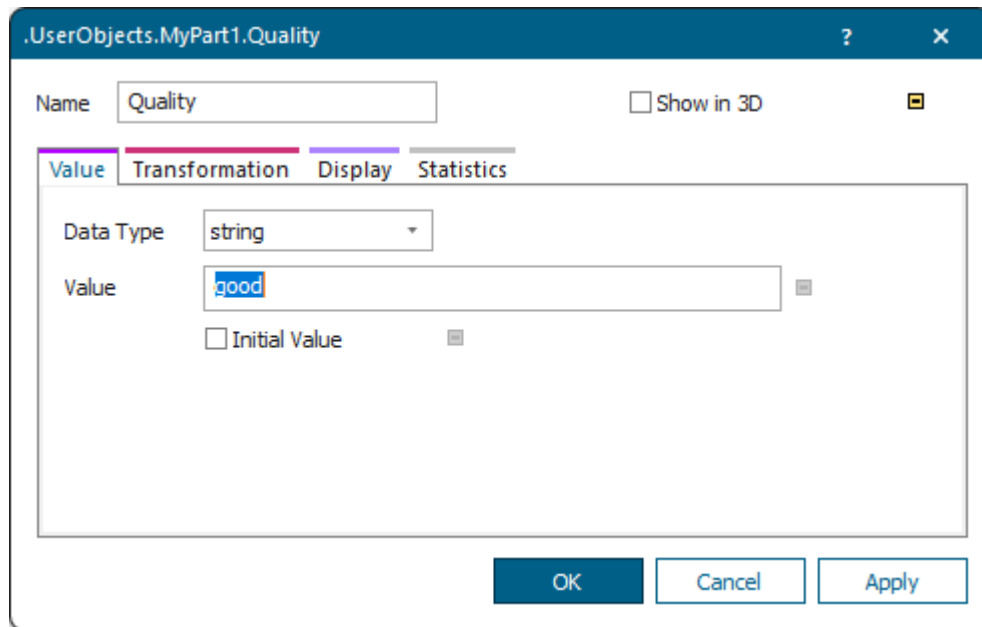
- Create the *user-defined attribute* with the name *Quality* for a part.
- Change its value with a *Method* to reflect a changed state, good or bad for example.
- Depending to its quality move the part either on to the next station or to a reworking station where it will be fixed to meet the quality criteria.

To create a *user-defined attribute* for the selected object:

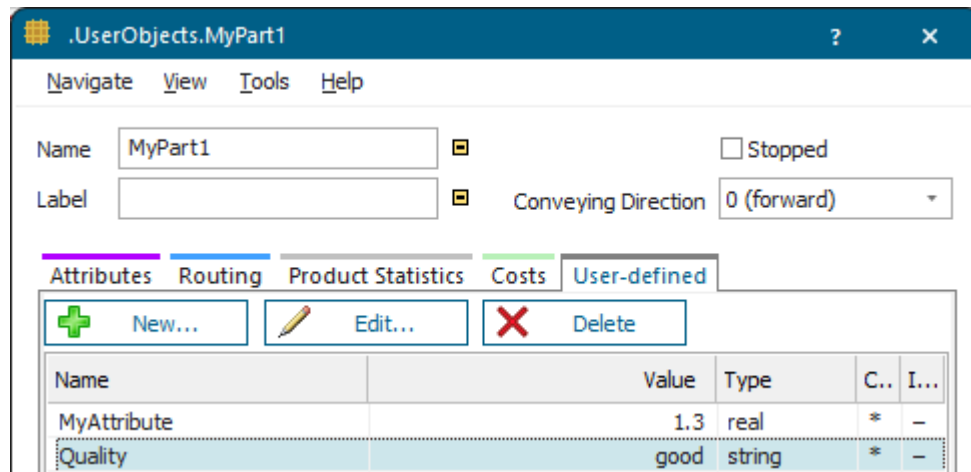
- Scroll to the tab **Tab User-defined** and click **New**  **New...**.



- Enter a meaningful name for your *user-defined attribute* into the text box. The name has to be unique, i.e., no other built-in or user-defined *attribute* or *method* may have the same name!
- Select a **Data Type**, compare **Data Types**.
- Enter a value into the text box that is compatible with this data type you selected.



- Click **OK** to add the *user-defined attribute* you just created. *Plant Simulation* sorts the attribute at the correct location in the alphabet into the list.



Method execution of *user-defined attributes of data type method*, which are deleted, while they are being executed is terminated immediately, it will not be continued! Compare the following examples:

- Suppose that a *MU* has a *user-defined attribute of data type method*, which was suspended via a *waituntil instruction*. Then, the *MU* is deleted in the *Drain*, which naturally also deletes its *user-defined attributes*. Thereupon the *Method* is deleted from the list of suspended methods and method execution is terminated, i.e., the instructions after the *waituntil instruction* will not be executed. If the *attribute method* has been called from another *Method*, execution of this *Method* will be continued. If the *attribute method* had a return value, the value, which the implicitly existing local variable *result* had at this point in time, will be returned to the calling *Method*.
- Suppose that a *MU* has a *user-defined attribute of data type method*, which is called by the **Entrance Control** of a *Station*. The *attribute method* in turn calls a *Method* that is inserted in the *Frame*. This *Method* deletes the *MU* with the statement `Station.cont.delete`. Both the execution of the *attribute method* and of the *Method* called by it are terminated immediately. The return value of the *attribute method* (or VOID if no return value was declared) will be returned to the **Entrance Control**, whose execution is continued.

Compare [User-defined Attributes \[general description\]](#)

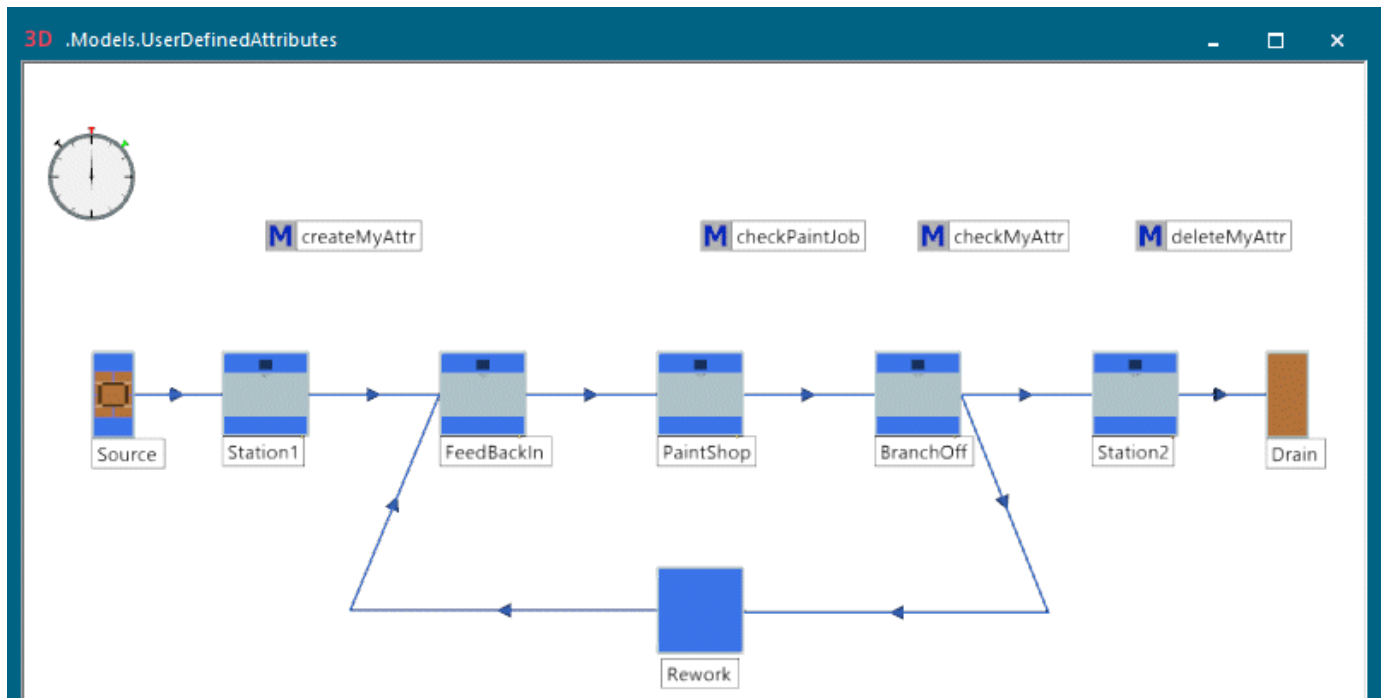
Back to [Customize the Behavior of Objects](#)

## Create a User-defined Attribute During the Simulation

Instead of manually creating a user-defined attribute by entering the required information on the tab **User-defined**, you can also dynamically create and delete *user-defined attributes* during the simulation by programming methods.

In our example we demonstrate how to create and delete a *user-defined attribute* for our part, i.e., a moving object. Depending on a statistical distribution parts with the *user-defined attribute* (property) *Bad* are removed from the material flow after the station *paint shop*, are moved to the *rework station* to be repainted, and are then fed back into the production line.

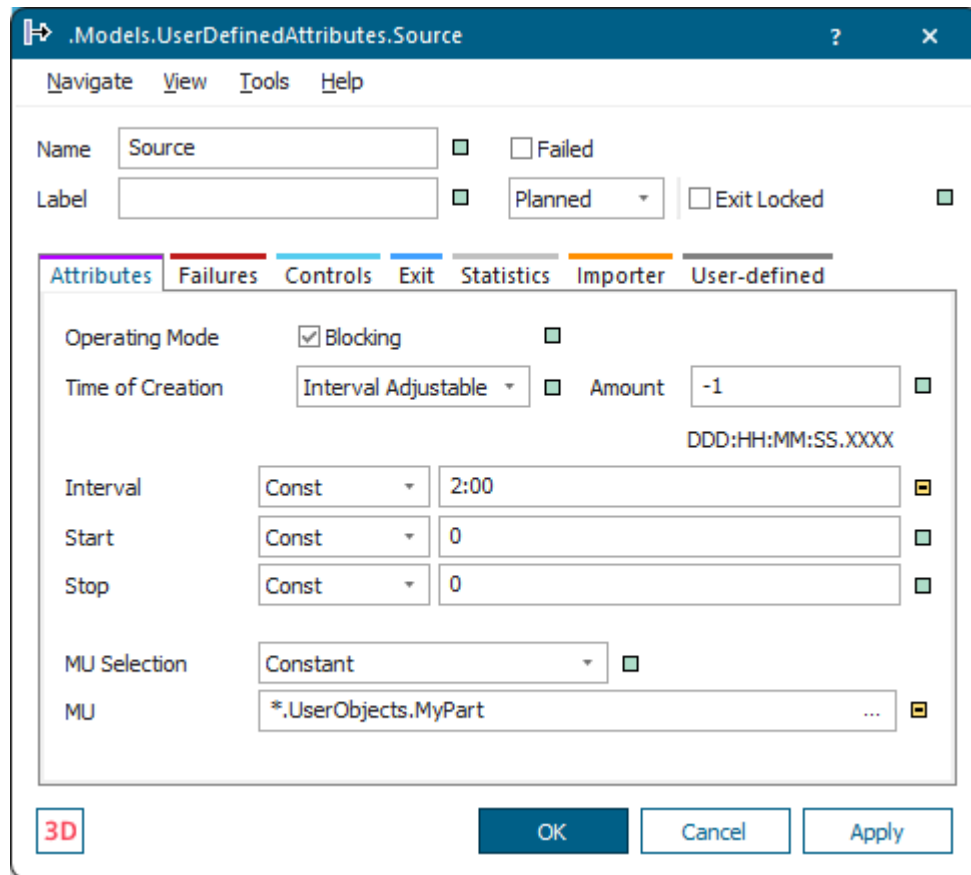
Our model looks like this:



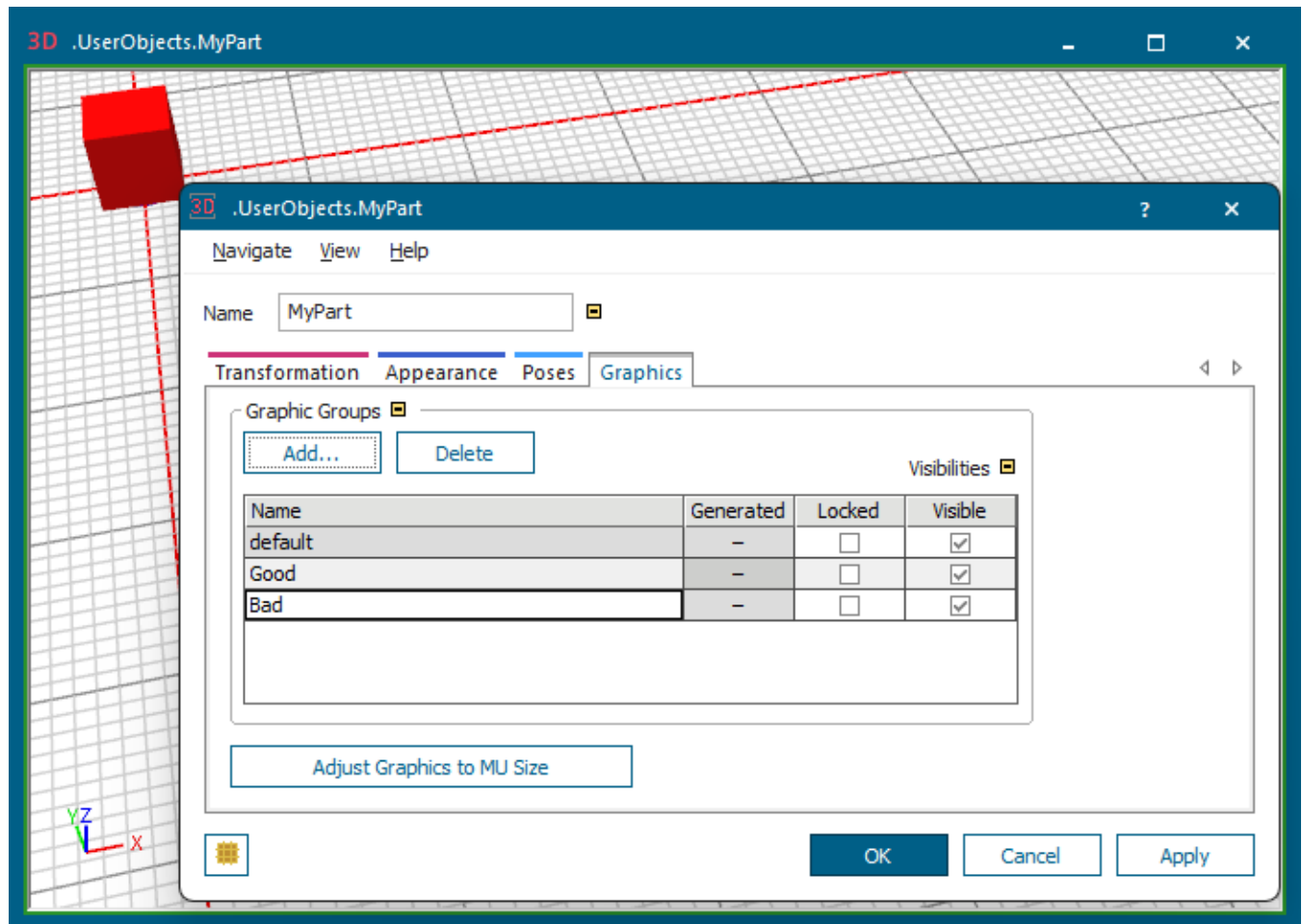
To create the model, proceed as follows:

- Insert the *Source* which creates the parts. In our example it produces a part of type *MyPart* every two minutes.

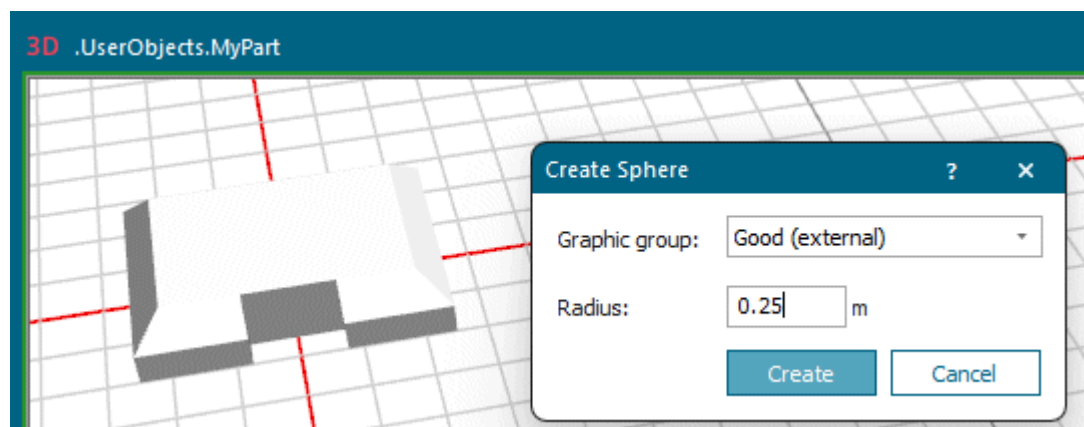




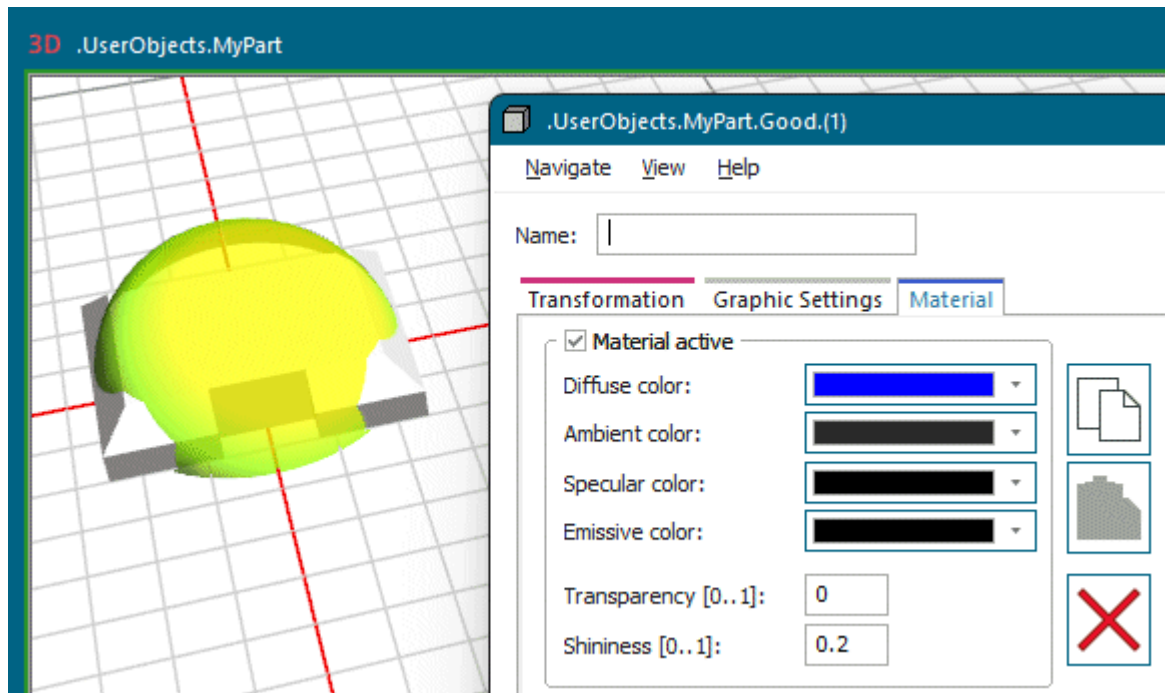
- To visualize the good and the bad parts during the simulation run, assign the properties **Good** and **Bad** each to the object. Then create the respective graphic groups and a graphic each for **Good** and **Bad**.
- Click the part named *MyPart* in the *Class Library* with the right mouse button and select **Open in 3D**.
- Click the background of the window and select **Edit 3D Properties**.
- Change to the tab **Graphics** and add a graphic group each and name them **Good** and **Bad**. Click **OK**.



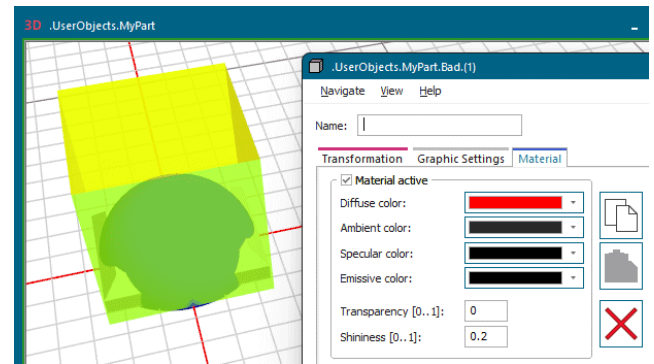
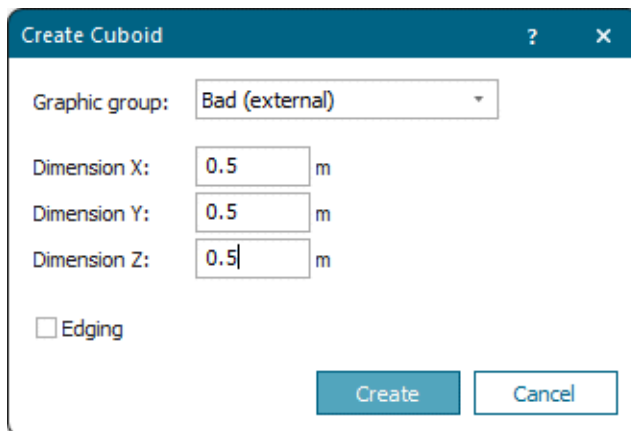
- Next, create a graphic to differentiate the good parts from the bad parts. Click the ribbon tab **Edit**. Select a **shape**.
  - We selected a **Sphere** for **Good** and then chose these settings.



- Then select a **Material** for the **Sphere**. We selected **Blue**.



- We selected a **Cuboid** with these settings for **Bad** and red as material.



- We finally have to adjust our method named *checkPaintJob* so that *Plant Simulation* recognizes the graphic groups.

```

var val := z_uniform(1,0,1)           // random number stream, lower
bound, upper bound                    // checks the value
if val < 0.1                           // bad paint job
  @.Paint := false                     // switch icon
  @.CurrIcon := "Bad"
else
  @.Paint := true                      // good paint job
  @.CurrIcon := "Good"                 // don't change icon
end
@._3D.VisibleGraphicGroups := [@.CurrIcon] // code to show graphic groups
in 3D

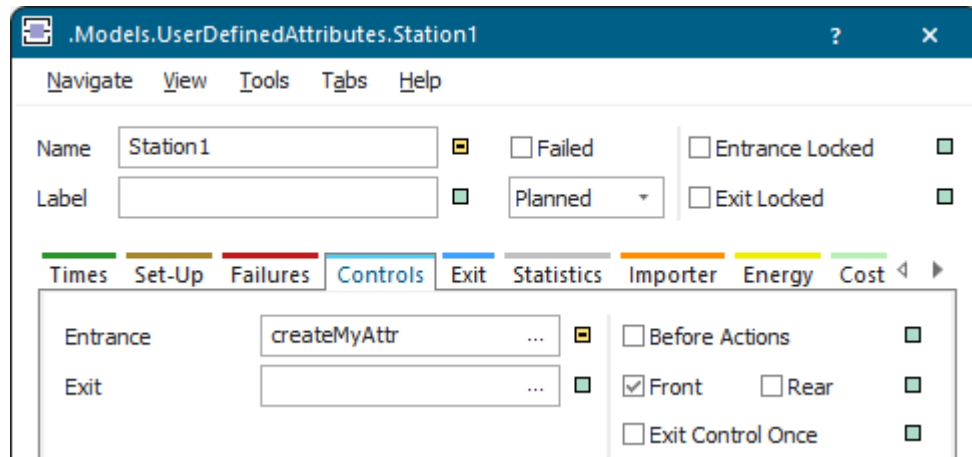
```

```
print "Value: ",val," : ",@.CurrIcon           // prints the value and Good
or Bad to                                       // the Console, depending on
the value
```

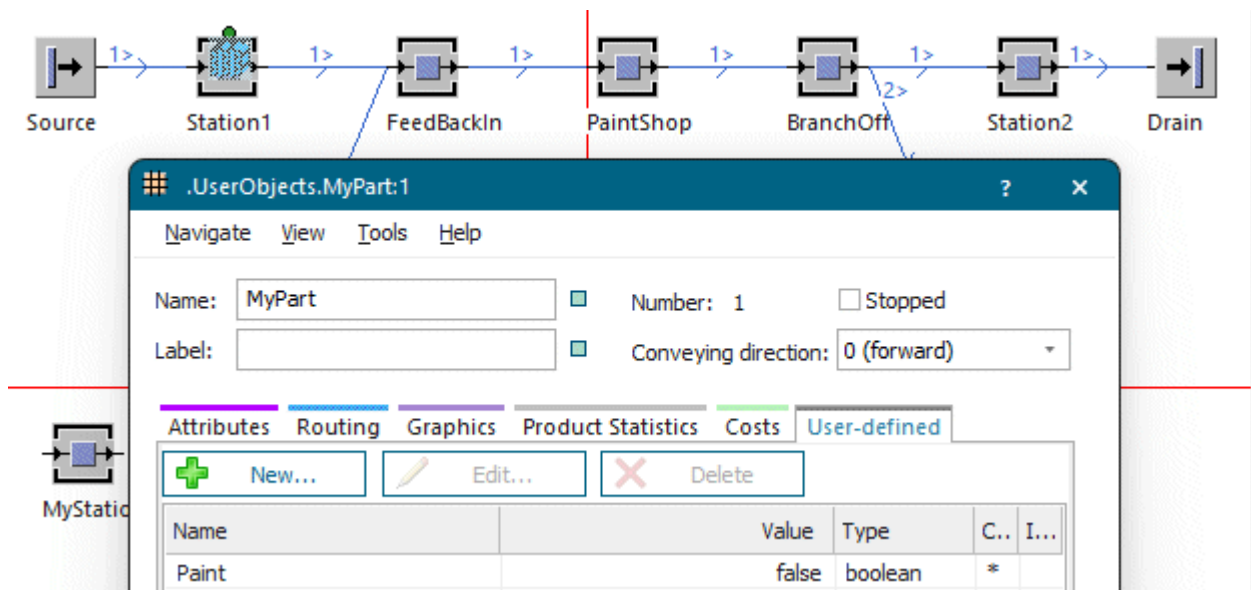
- To dynamically create the *user-defined attribute* of the part when it reaches *Station1*, you have to program a method. We named our method *createMyAttr*. It creates the *user-defined attribute* named **Paint**. The source code looks like this:

```
@.createAttr("Paint","boolean")
```

- Then enter this method as the **Entrance Control** into *Station1*.



To make sure that the method works as intended, start the simulation, and double-click the icon of the first part. Click the tab **User-defined** and check if it shows the attribute **Paint**.



- Then the station *PaintShop* has to check the paint job of the painted part and decide if it is good or bad. To accomplish this, you have to program a method.

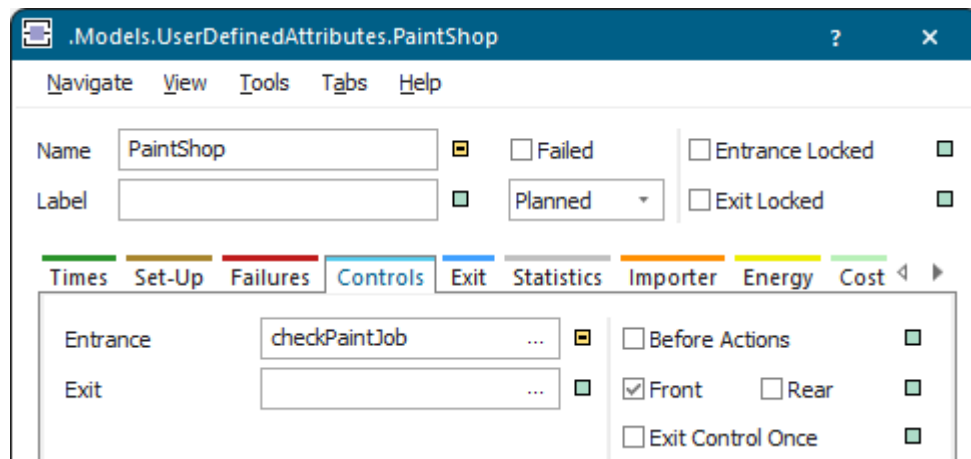
We named our method *checkPaintJob* and entered the following source code:

```
var val := z_uniform(1,0,1)           // random number stream, lower
bound, upper bound
if val < 0.1                          // checks the value
  @.Paint := false                   // bad paint job
  @.CurrIcon := "Bad"               // switch icon
else
  @.Paint := true                   // good paint job
  @.CurrIcon := "Good"              // don't change icon
end
print "Value: ",val," : ",@.CurrIcon // prints the value and Good or
Bad to                               // the Console, depending on the
value
```

The distribution *Z\_uniform(1,0,1)* creates values between 0 and 1.

The *if statement* checks this value. If it is smaller than 0.1, the attribute **Paint** is set to **false** and the icon of the part changes to **Bad**. If the value is greater than 0, the attribute remains **true** and the icons is **Good**. The **uniform distribution** thus creates one **bad** part for each ten **good** parts.

We then entered this method as the **Entrance Control** into the *PaintShop*.

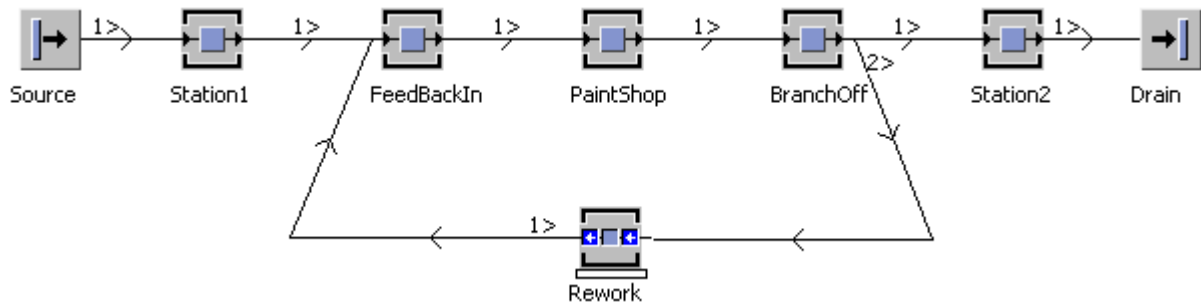


- Next, the station *BranchOff* has to check the *user-defined attribute* named **Paint**. To accomplish this, you have to program a method. We named our method *checkMyAttr* and entered the following source code:

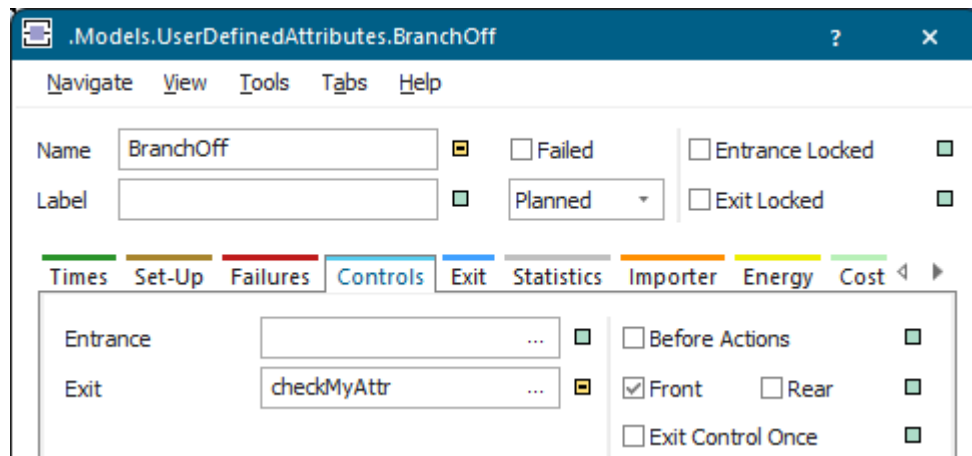
```
if @.Paint // checks the user-defined attribute
  @.move(1) // moves the part along on Connector1
else
  @.move(2) // moves the part along on Connector2
end
```

If the value of the *user-defined attribute* **Paint** is **true**, the part is moved along the *Connector* with the number **1**. If the value is **false**, it is moved along the *Connector* with the number **2**.

If the quality is **good**, the part is moved on to *Station2*, if it is **bad**, it is moved to the *Rework* station, where it is fixed and it is then fed back into the line.



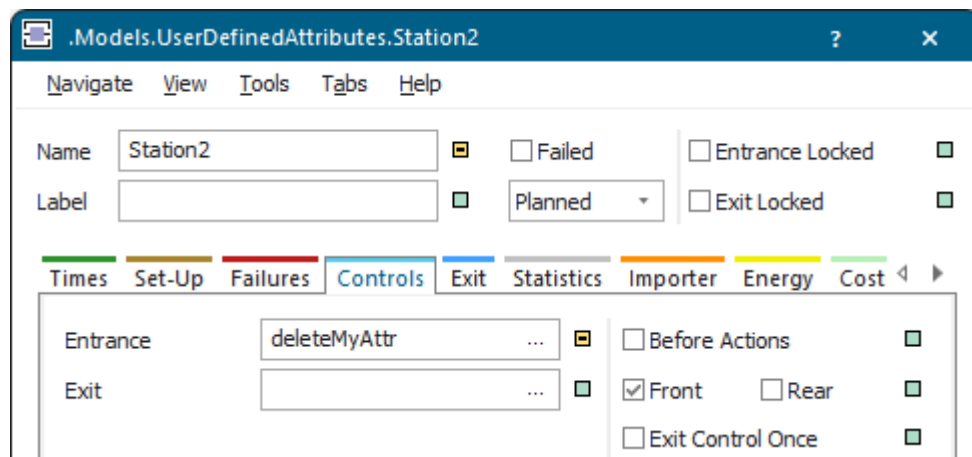
Then, enter this method as the **Exit Control** into the station *BranchOff*.



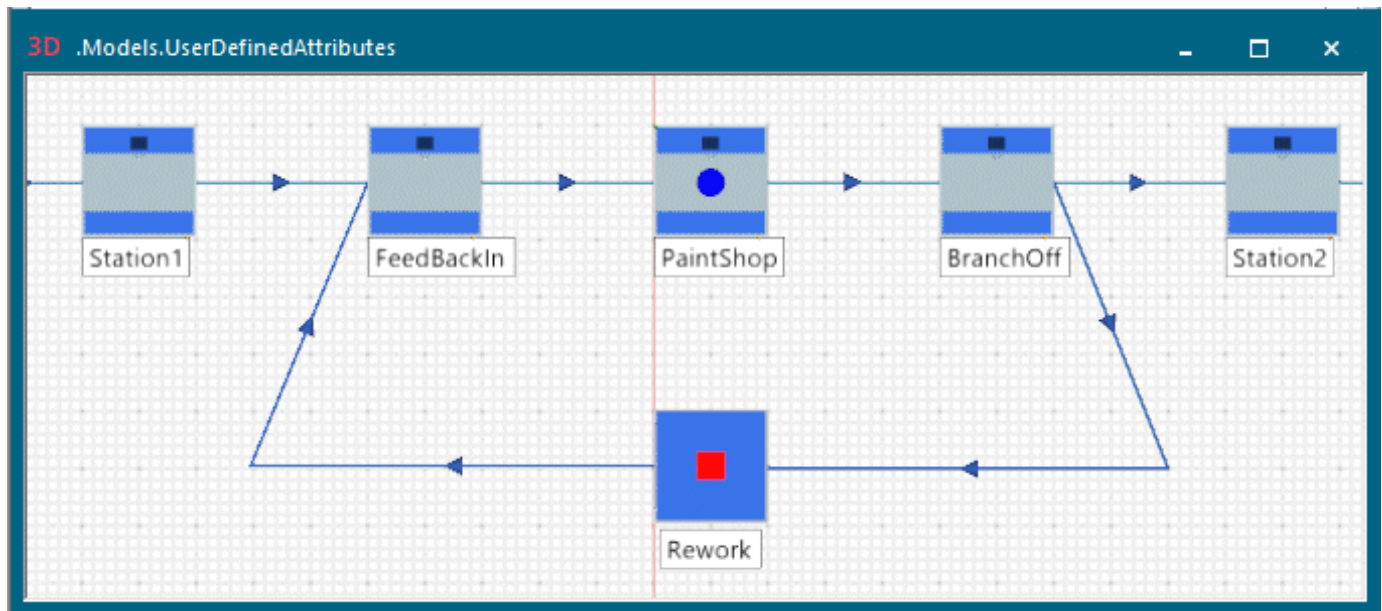
- Finally, program the method that deletes the *user-defined attribute Paint*, as it is not needed any more. We named our method *deleteMyAttr* and entered the following source code:

```
@.deleteAttr("Paint")
```

- Then, enter this method as the **Entrance Control** into *Station2*.

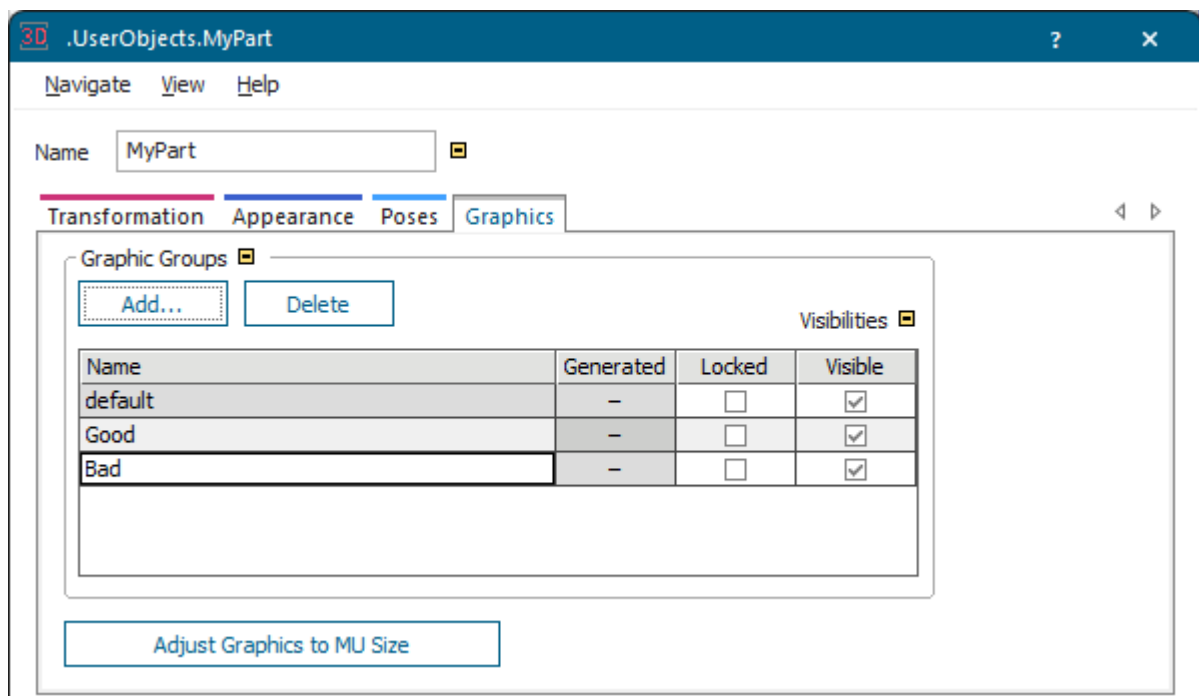


The result looks like this:

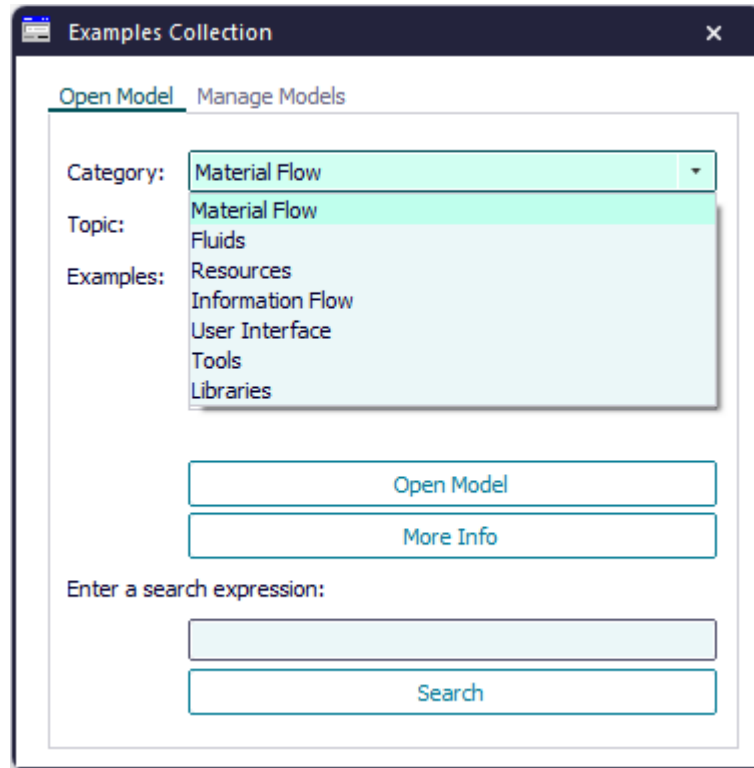


When we run the simulation, we see that the *Source* already produces good and bad parts.

As the color is only applied to the part in the *PaintShop*, we clear **Visible** for the graphic groups named **Good** and **Bad**. The *Source* then produces the part with the neutral standard graphic.



Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



Compare [User-defined Attributes \[general description\]](#)

Back to [Customize the Behavior of Objects](#)

## Create a Tooltip as a User-defined Attribute

At times you might want to create a *tooltip* for certain objects or for *sub-Frames* in your simulation model.

To do so, proceed as follows:

- Double-click the respective object in your *simulation model* and change to the **Tab User-defined**.
- Click **New**  **New...**
- Type in **Tooltip** as the **Name** and select the **Data Type > string**.  
Type in the text you want to show as the *tooltip* into the text box **Value**.

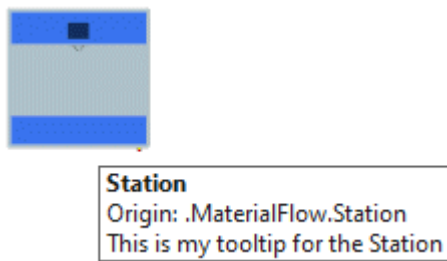


- Click **Apply** and click the object in the *3D window* with the right mouse button. Check if the *tooltip* meets your needs.

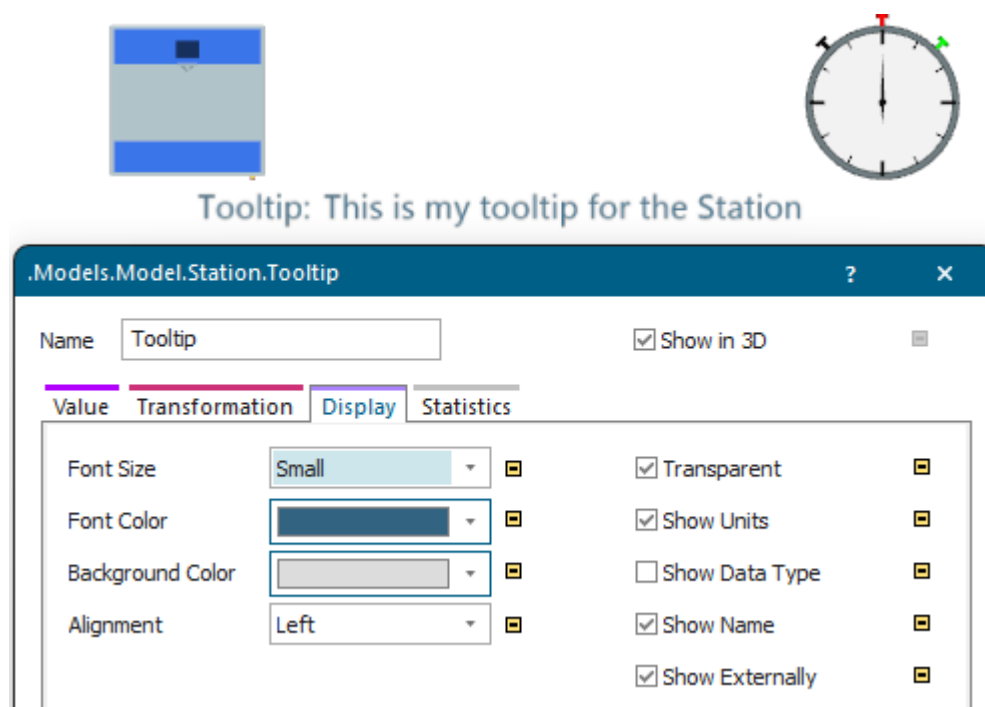
The **standard tooltip** always shows the name of the object in bold face in the first row, **Station** in our case.

This is followed by the origin of the object, **Origin: .MaterialFlow.Station** in our case.

The third row shows the *tooltip* we defined, **This is my tooltip for the Station** in our case.



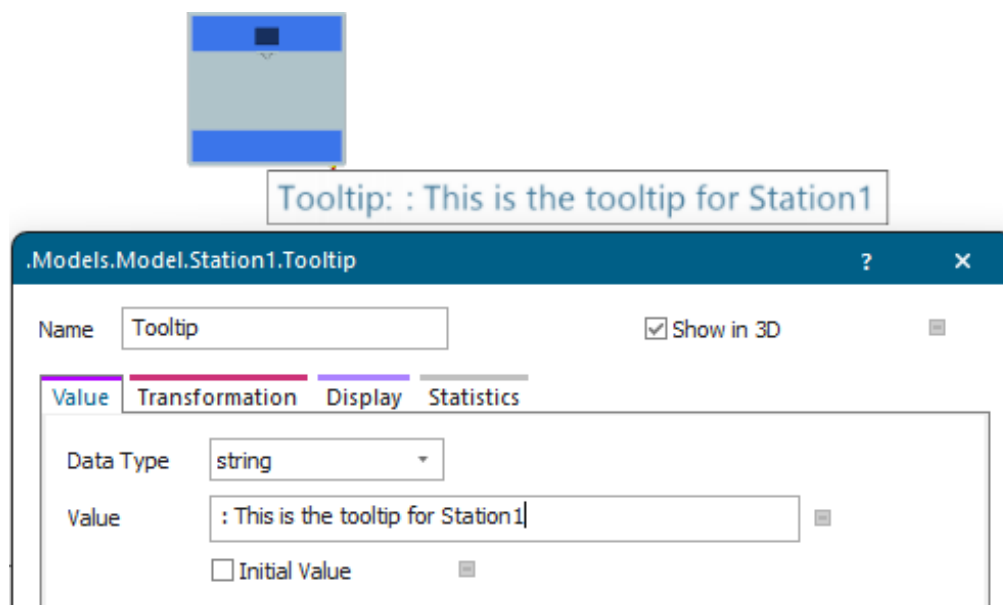
- You can also select to **Show the tooltip in 3D** and select **Transformation** and **Display** settings.



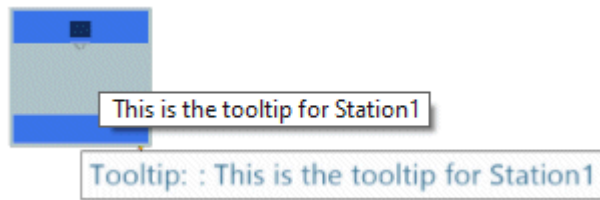
- Click **OK** once you are satisfied with the results.

To only show the text of the *tooltip* you define, without the **standard tooltip** information that our programmers defined, proceed as described above.

- Enter a colon in front of the text you want to show as the *tooltip* into the text box **Value**.

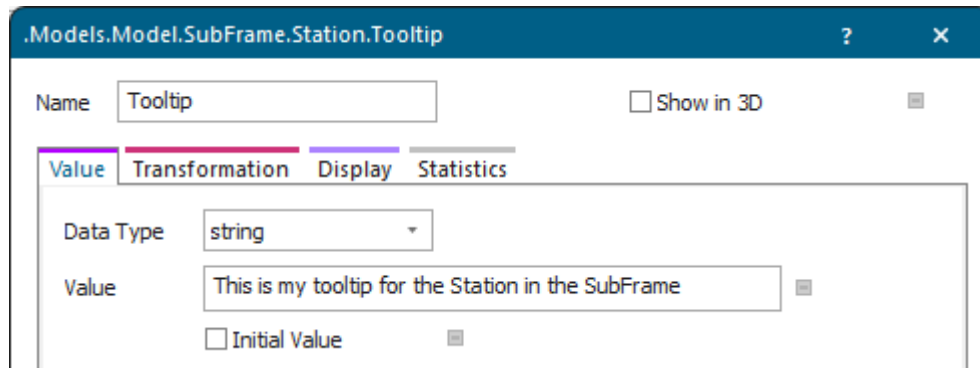


- Click **Apply** and click the object in the *3D window* with the right mouse button. Check if the *tooltip* meets your needs.

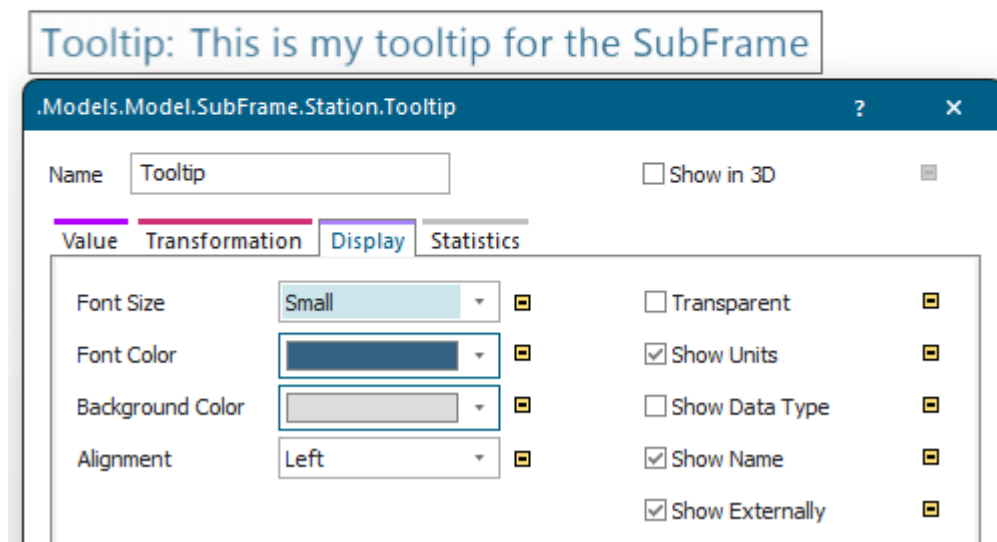


- Click **OK** once you are satisfied with the results.

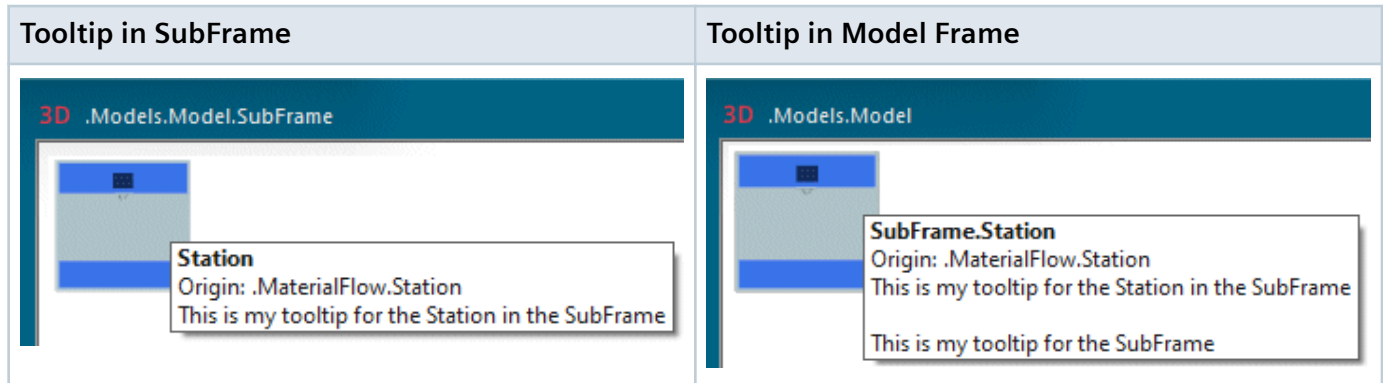
To create a *tooltip* for a *sub-Frame*, which you inserted into your *3D window*, proceed as described above.



To show the *tooltip* in the *3D window*, select **Show Externally**.



To create a *tooltip* for an object in a *sub-Frame*, which you inserted into your *3D window*, proceed as described above.



Compare [User-defined Attributes \[general description\]](#)

Compare [Show Tooltips of Objects in the Frame](#)

Back to [Customize the Behavior of Objects](#)

## Creating Entrance and Exit Controls

### Create Entrance and Exit Controls

You can modify the built-in transfer behavior of the material flow objects by programming an **Entrance Control** and/or an **Exit Control**.

The material flow object calls the *Method*, whose name you entered, as soon as a *MU* intends to enter or exit the respective object. Be aware that these controls override the standard transfer behavior, meaning that you yourself have to make sure that the *MU* moves to the correct station!

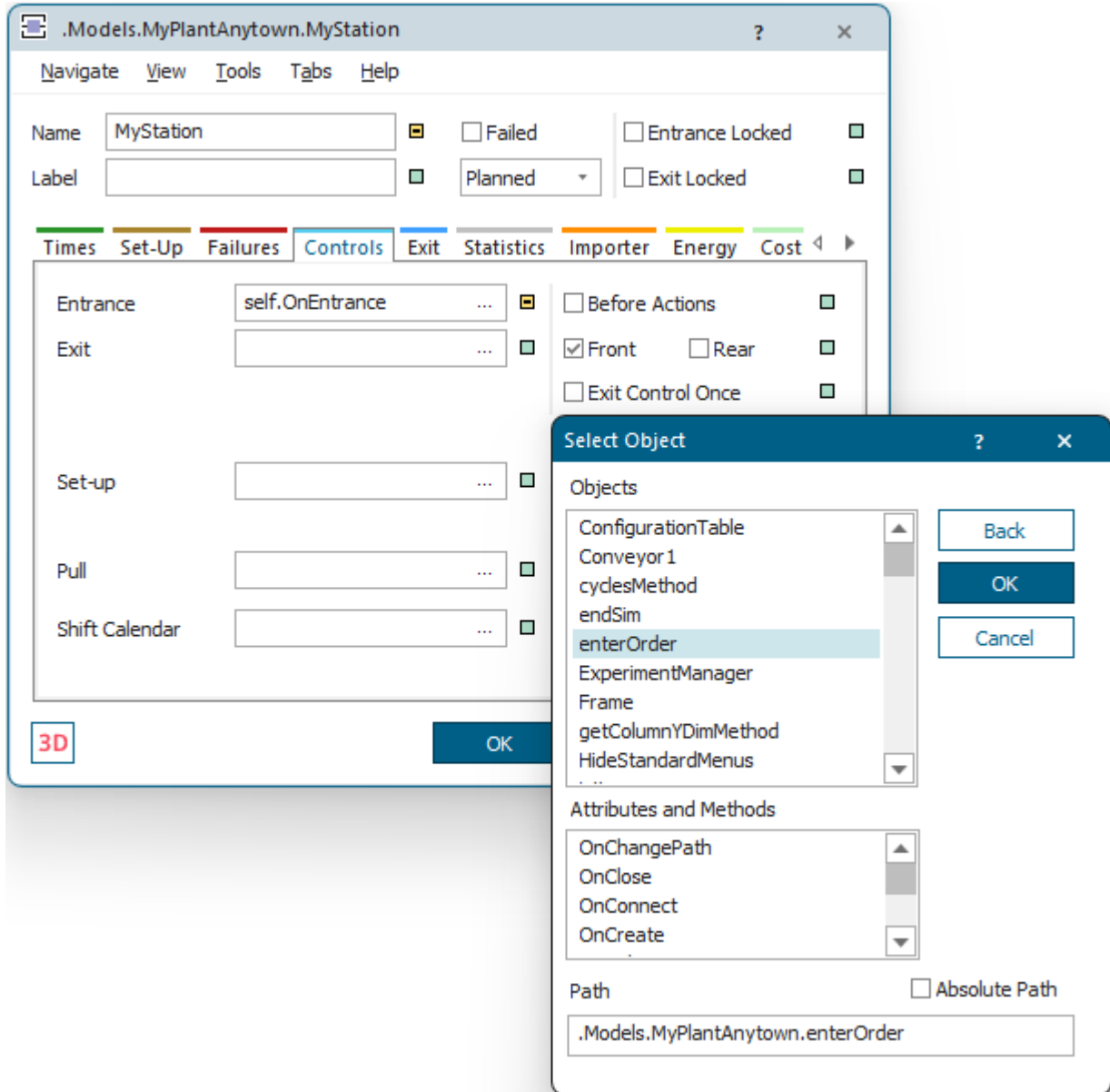
You can use *control methods* in a number of ways. You might, for example, count the number of incoming *MUs* and then, after a specified number has been reached, move the following *MUs* to a different successor.

You will program an **Entrance Control** and/or an **Exit Control** in a *Method*. This can either be:

- An object of type **Method M**, which can be located in a folder in the *Class Library* or in a *Frame*.  
If you want a number of objects, which are located in the same *Frame*, to call the same *Method*, you typically use a *Method* object, which you insert into that *Frame*.  
If you want a number of objects, which are located in different *Frames*, to call the same *Method*, you typically use a *Method* object, which you place in the *Class Library*.
- A **user-defined attribute of data type method**, which only the object to which you added it, can use. When you want all instances of a class to provide the same control, you typically create that control as a user-defined attribute in the class.

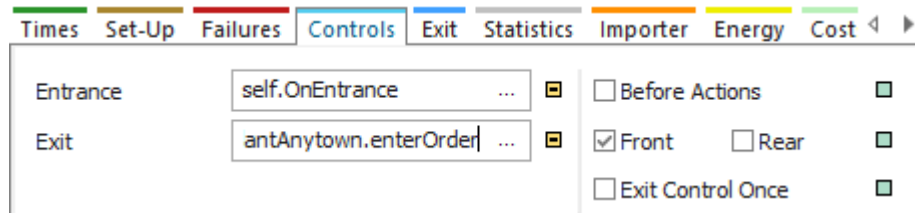
To tell *Plant Simulation* which *Method* to use, do one of the following. The process is the same for **Entrance** and **Exit** and **Backward Entrance** and **Backward Exit Controls**:

- Click the button  and select a *Method* in a *Frame* in the dialog **Select Object**. By default *Plant Simulation* enters the **relative path** into the text box.



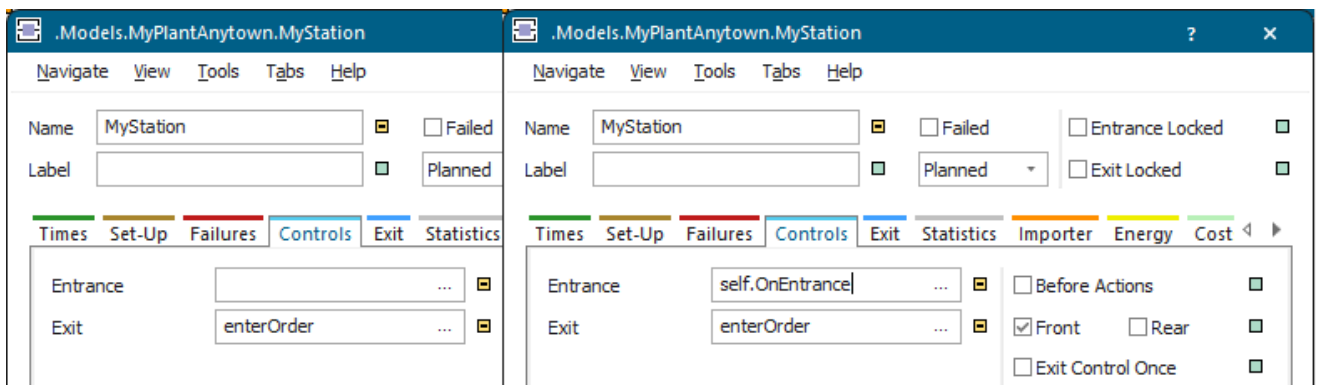
To activate the entrance control of the **point-oriented objects before** the standard actions are started, when a part enters the object, select **Before Actions**. Standard actions are, for example, starting the **Processing Time** or the **Set-up Time**, requesting **Services** through the *importer*, or changing the **assembly table** of the *AssemblyStation*. You can then, for example, change the **Processing Time** in the **Entrance Control**, which affects processing of the entering part.

- Select the *Method* in a *Frame*, hold down **Shift** and **Ctrl**, and drag it to the text box and drop it there. Here *Plant Simulation* enters the **absolute path** into the text box.



To open the dialog of the object, whose name you entered into the text box, either press **F2** or right-click the text box and select **Open Object**.

- Right-click into the text box and select **Create Control** on the context menu to create a *control method* as a *user-defined attribute* of this object.
  - Type a meaningful name into the text box and select **Create Control**. *Plant Simulation* then inserts `self.Name_you_typed_in_for_the_control`, such as `self.ExitControlEnginePlant`. Or
  - Select **Create Control**. *Plant Simulation* then inserts `self.OnBuilt_in_name_of_the_control`, such as `self.OnEntrance`.

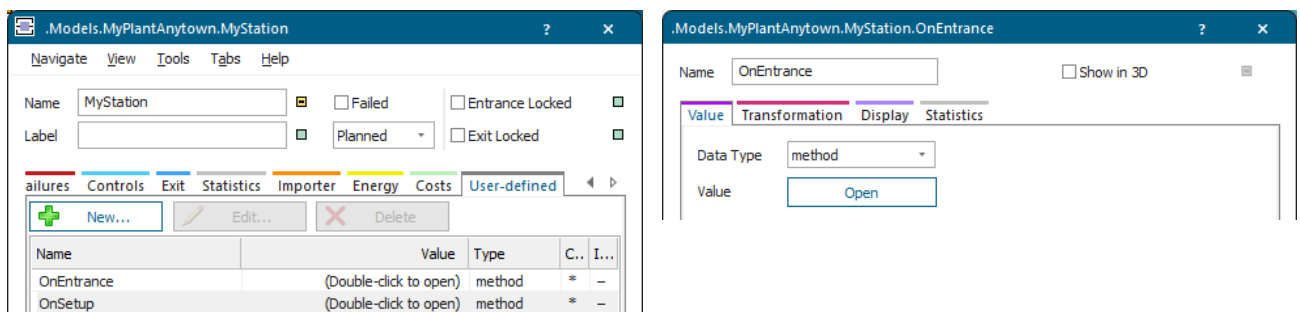


Type the source code of this control into the *Method* that opens.

To open the *Method editor*, either press **F2** or click the right mouse button into the text box and select **Open Object** and select the object or press **Shift** and double-click in the text box.

Be aware that a control you create with **Create Control** is a *user-defined attribute* of the respective object, not a *Method* object, which you can open in the *Frame*.

To open the control from the tab **User-defined** double-click it and then click **Open**.



- To delete this control, delete the *user-defined attribute*. Deleting the name from the text box just deletes the name, but keeps the *user-defined attribute*.

**Note**

The **Exit Control** can have an optional parameter of data type *object*. When the *Method* has this parameter, then the successor object, which pulls the *MU* because the *MU* is contained in the blocking list of the object, will be assigned to this parameter. The *MU* will be pulled, for example, when the successor becomes available again or when its entrance is opened.

You can:

- [Define Controls for Point-Oriented Objects](#)
- [Define Controls for Length-Oriented Objects](#)

Compare [Transfer Parts from Station to Station](#)

Compare [Create a Sensor \[material flow objects\]](#)

Compare [Customize the Behavior of Objects](#)

Back to [Model the Flow of Materials, Advanced](#)

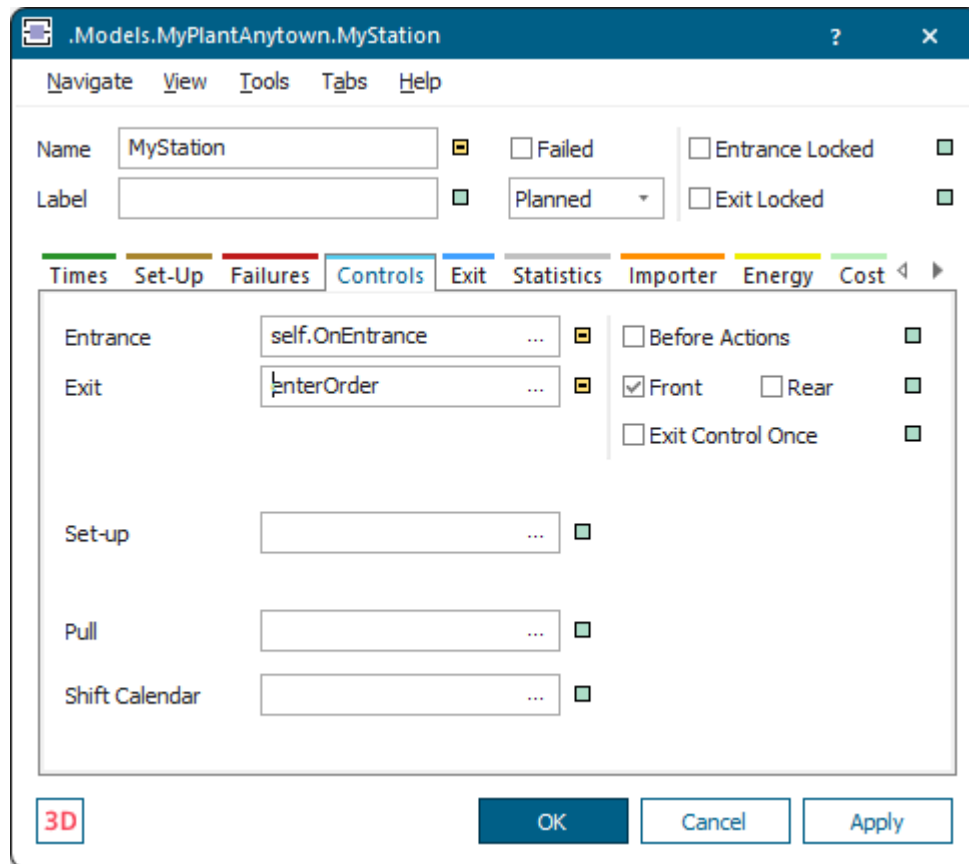
## Defining Controls for Point-Oriented Objects

### Define Controls for Point-Oriented Objects

You can define controls for point-oriented objects,

**Point-oriented objects** are the material flow objects that provide one or more processing stations, but do not have a length and do not take the length of the *MU* into account.

The *Station* , the *ParallelStation* , the *AssemblyStation* , the *DismantleStation* , the *Buffer* , the *PlaceBuffer* , the *Store* , the *Sorter* , the *Source* , and the *Drain*  are point-oriented.



Plant Simulation activates the **Entrance Control** of the **point-oriented objects**, such as the *Station* , once the *MU* has entered the object in its entirety. This is the default setting for which the check box   
 Entrance   ☐ Before Actions  is cleared.

To activate the **entrance control before the standard actions are started**, when a part enters the object, select **Before Actions**. Standard actions are, for example, starting the **Processing Time** or the **Set-up Time**, requesting **Services** through the *importer*, or changing the **Assembly Table** of the *AssemblyStation*. You can then, for example, change the **Processing Time** in the **Entrance Control**, which affects processing of the entering part.

Plant Simulation activates the **Exit Control**, when a *MU* exits the object.

The check boxes **Front** and **Rear** set, when the *MU* triggers the *Method*.

- Select **Front** to activate the control as soon as the *MU* is ready to exit the object. The **Exit Control** then has to move the part to another object, as the built-in behavior of the material flow object, i.e., moving the *MU* on to the successors along the *Connectors*, has been overridden by the **Exit Control**.

#### Note

The same *MU* can call the **front-triggered exit control** more than once, if that *MU* could not exit the object and entered itself into the **Blocking List** of the destination object. As soon as the



destination object is ready to receive the *MU*, the *MU* is assigned a new **Out** event, which triggers the **front-triggered exit control** again.

Select **Rear** to activate the control as soon as the rear of the *MU* has completely exited the object. This control does not override the built-in behavior of how *MUs* move to the successor of the current object.

#### Note

*Plant Simulation* calls the **rear-activated exit control** once only. It does not override the default exit strategy.

#### Note

Compare the **Out**, **OutEnd** and **Reference** events in the [List](#) of scheduled events.

- The check boxes **Front** and **Rear** are not mutually exclusive. If you **Front** and **Rear**, *Plant Simulation* executes the respective control in both cases.

You can, for example:

- [Change the Processing Time in the Entrance Control](#)
- [Change the Importer Services in the Entrance Control](#)
- [Change the Assembly Table in the Entrance Control](#)
- [Distribute Parts with an Exit Control](#)

Compare [Define Controls for Length-Oriented Objects](#)

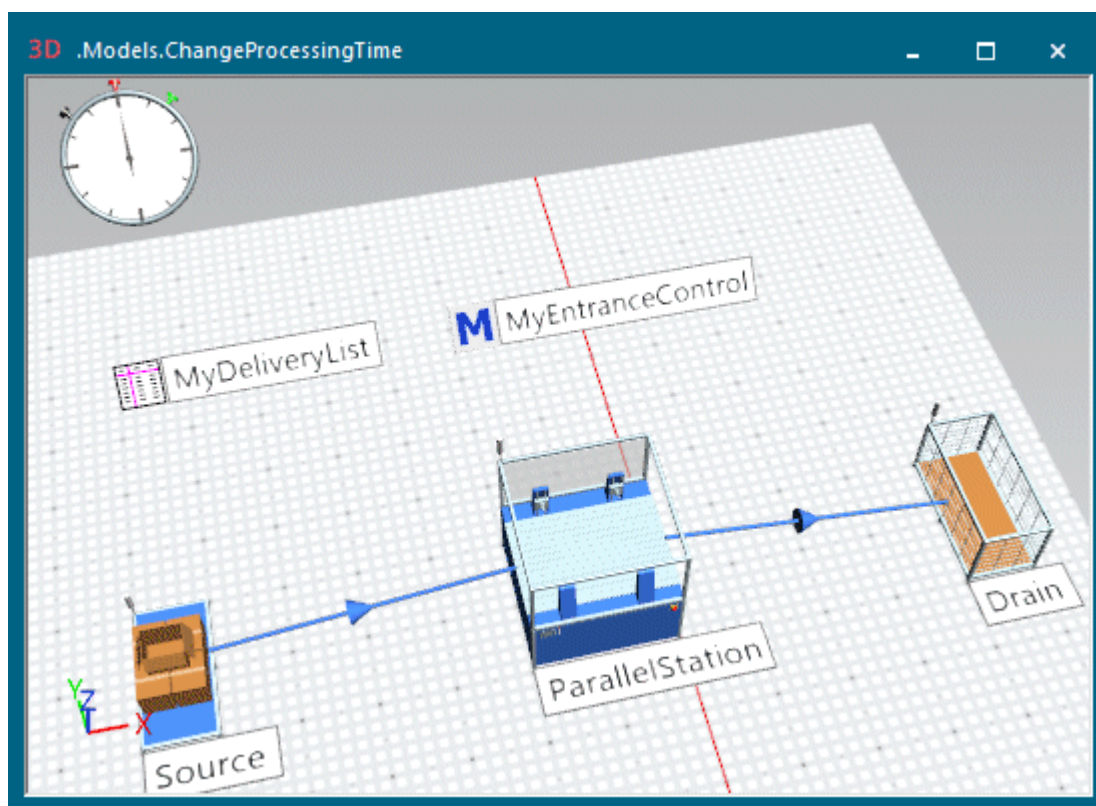
[Back to Create Entrance and Exit Controls](#)

## Change the Processing Time in the Entrance Control

You can change the **Processing Time** of a material flow object, a *ParallelStation* in our case, in the **Entrance Control**. It can then determine how the part that is already located on the object is being processed.

To activate the **Entrance Control** of a **point-oriented object** before the standard actions are started, when a part enters the object, select **Before Actions**.

The finished simulation model to demonstrate this feature looks like this:

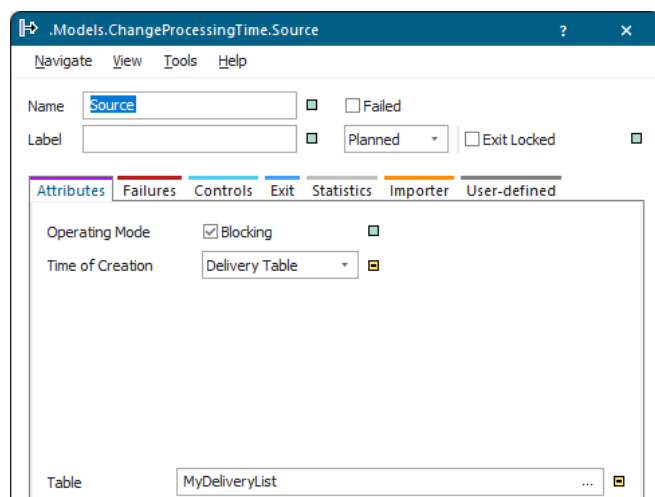


To create the model, proceed like this:

- Insert and configure the *Source* which produces the parts:

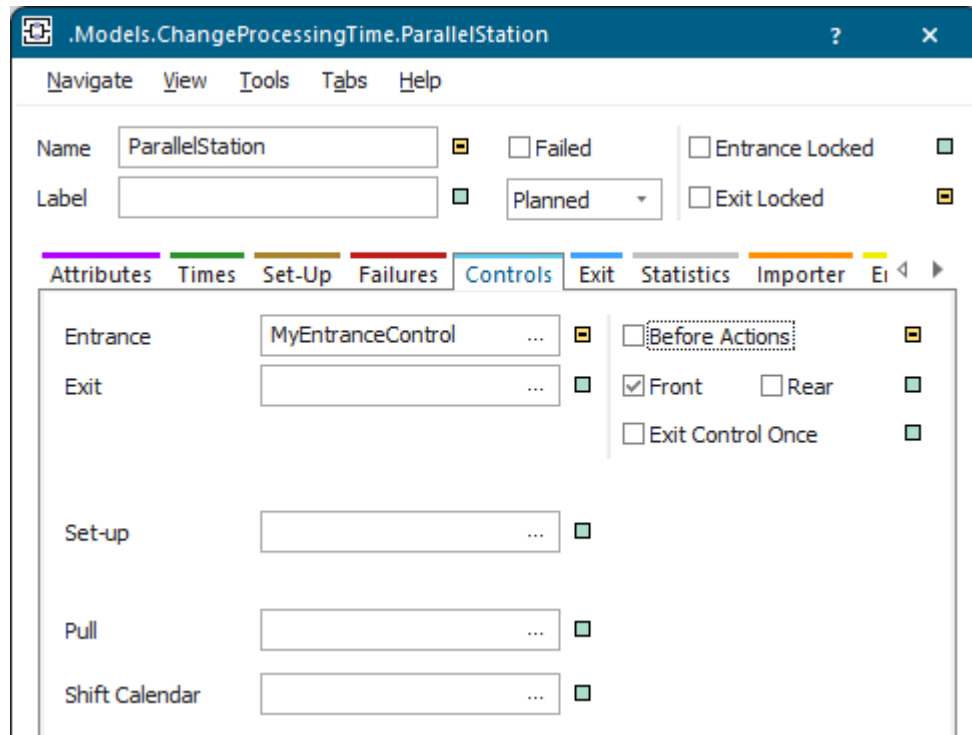
Select the **Time of Creation**. We selected **Delivery Table** and then selected the name of the **Delivery Table**, which we inserted before, namely the *DataTable* named *MyDeliveryList*. Selecting the table automatically applied the correct format.

- Enter the required data into the **Delivery Table**. We typed in the data shown in the screenshot below.



|        | time          | object    | integer | string | table      |
|--------|---------------|-----------|---------|--------|------------|
|        | 1             | 2         | 3       | 4      | 5          |
| string | Delivery Time | MU        | Number  | Name   | Attributes |
| 1      | 1:00.0000     | .MUs.Part | 1       | PartA  |            |
| 2      | 2:00.0000     | .MUs.Part | 2       | PartB  |            |
| 3      |               |           |         |        |            |

- Insert and configure the station which processes the parts. We used a *ParallelStation*. We entered a **Processing Time** of 2 minutes, i.e., 2:00, on the tab **Times**. We selected the method *MyEntranceControl*, which we inserted before, as the **Entrance Control**. We did not change any of the other settings.

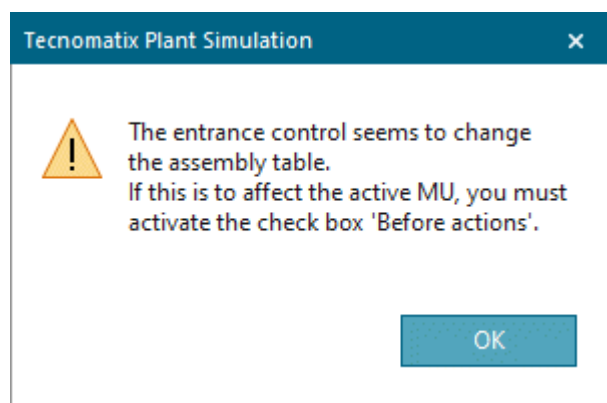


Then, program the **Entrance Control**. We typed in the following instructions, which change the **Processing Time** of the part called PartA to 10 minutes. The part called PartB will be processed for 2 minutes, i.e., for the time which we entered as the **Processing Time**.

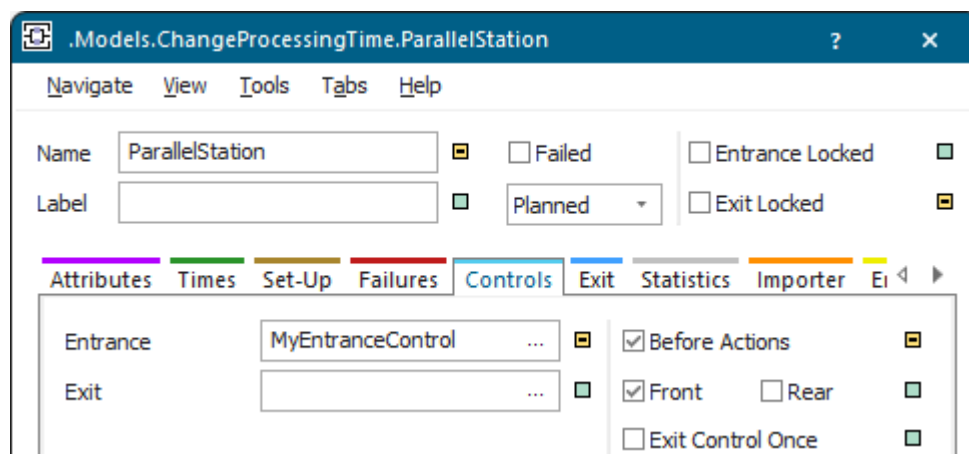
The source code looks like this:

```
switch @.Name
case "PartA"
    ?.ProcTime := str_to_time("10:0") // PartA will be processed for 10
minutes
case "PartB"
    ?.ProcTime := str_to_time("2:0")  // PartB will be processed for 2
minutes
end
```

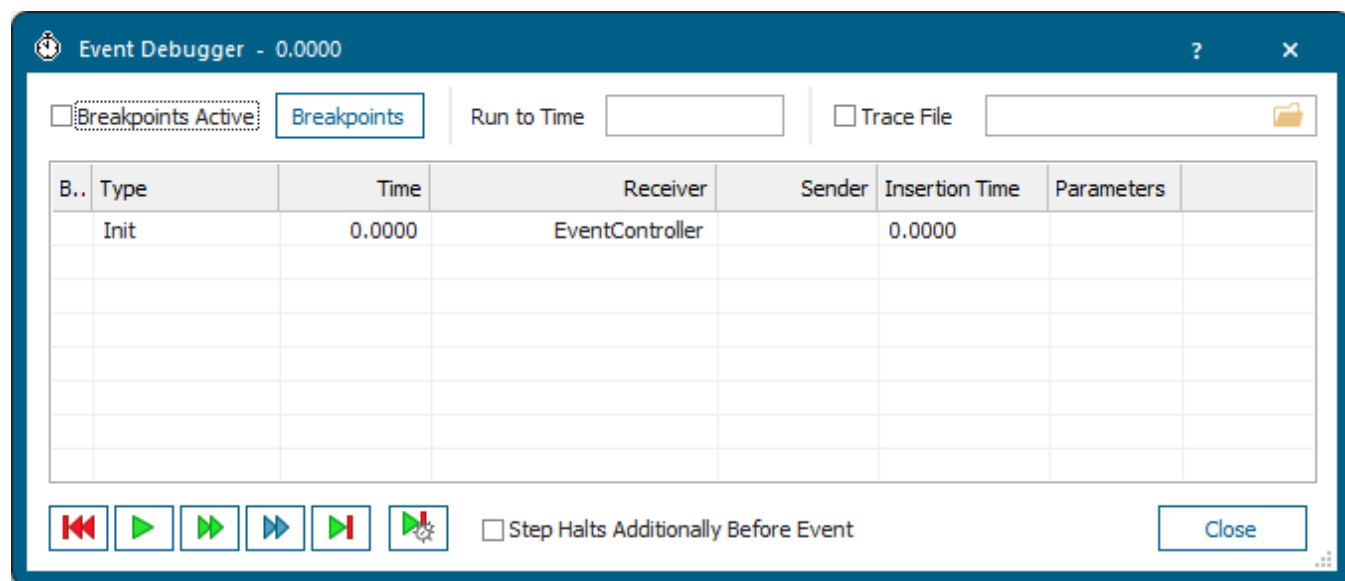
When you click **Apply**, *Plant Simulation* shows a message telling you that you have to activate the check box **Before Actions**, if the instructions in the control are to apply to the active part.



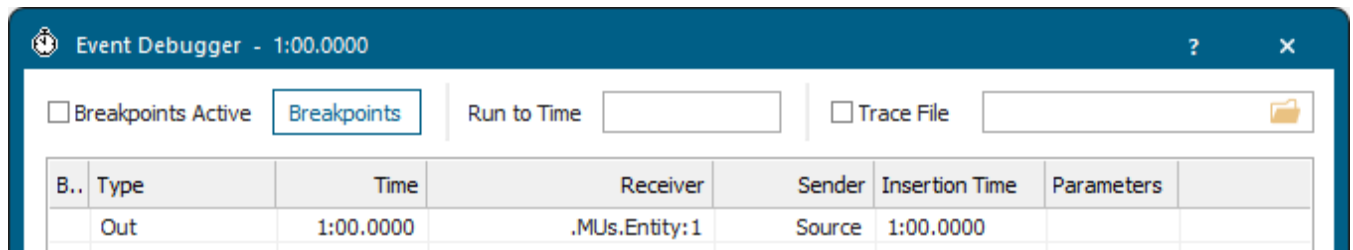
Select the check box.



Run the simulation and watch what happens. To follow how the events are processed during the simulation, you can click  in the *EventController* and then click  in the *EventDebugger*. At the start of the simulation the *Source* creates a part, signified by the **CreateMU** event.



When you click  again, the part leaves the *Source*, signified by the **Out** event.

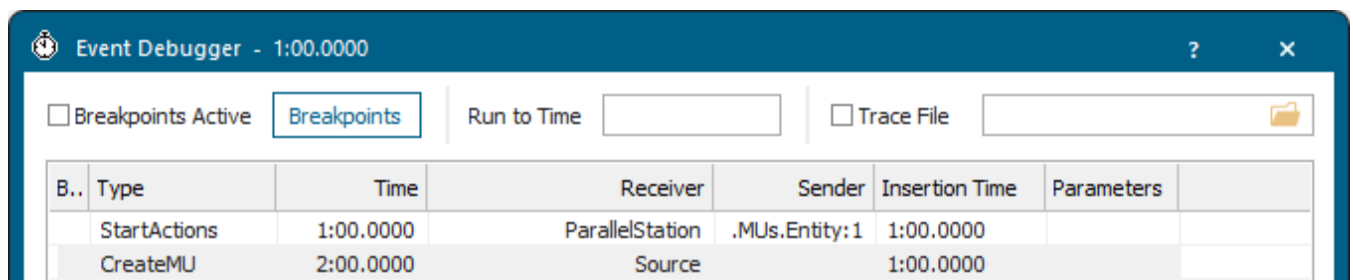


Event Debugger - 1:00.0000

☐ Breakpoints Active **Breakpoints** Run to Time  ☐ Trace File

| B.. | Type | Time      | Receiver       | Sender | Insertion Time | Parameters |
|-----|------|-----------|----------------|--------|----------------|------------|
|     | Out  | 1:00.0000 | .MUs.Entity: 1 | Source | 1:00.0000      |            |

When you click  again, the part is located on the *ParallelStation* and the **Entrance Control** is activated before the standard action, in this case setting the **Processing Time**, is started. This is signified by the **StartActions** event.

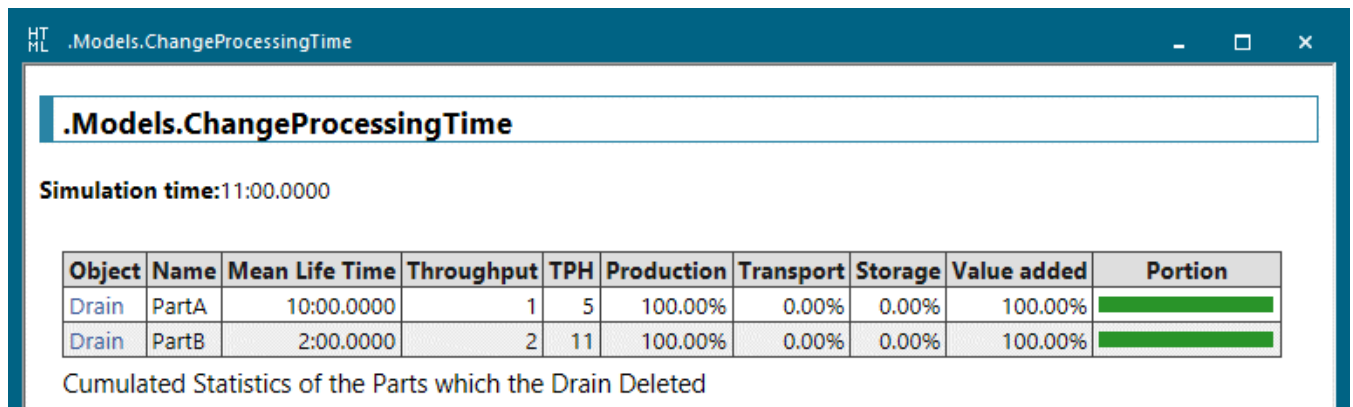


Event Debugger - 1:00.0000

☐ Breakpoints Active **Breakpoints** Run to Time  ☐ Trace File

| B.. | Type         | Time      | Receiver        | Sender         | Insertion Time | Parameters |
|-----|--------------|-----------|-----------------|----------------|----------------|------------|
|     | StartActions | 1:00.0000 | ParallelStation | .MUs.Entity: 1 | 1:00.0000      |            |
|     | CreateMU     | 2:00.0000 | Source          |                | 1:00.0000      |            |

The **mean life time** is **10 minutes** for **PartA** and **2 minutes** for **PartB** as we programmed in our control.



HT .Models.ChangeProcessingTime

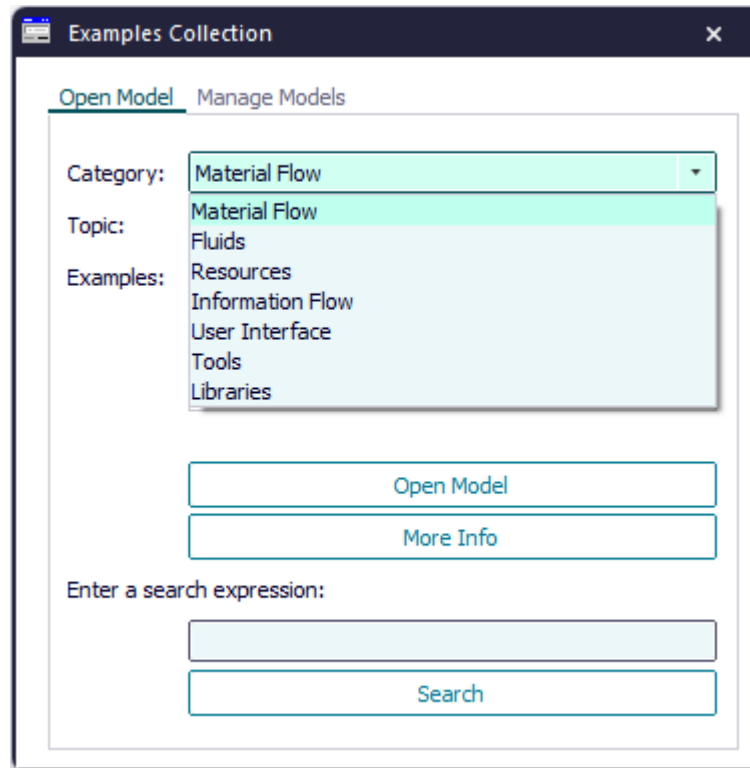
**.Models.ChangeProcessingTime**

Simulation time: 11:00.0000

| Object | Name  | Mean Life Time | Throughput | TPH | Production | Transport | Storage | Value added | Portion     |
|--------|-------|----------------|------------|-----|------------|-----------|---------|-------------|-------------|
| Drain  | PartA | 10:00.0000     | 1          | 5   | 100.00%    | 0.00%     | 0.00%   | 100.00%     | <div></div> |
| Drain  | PartB | 2:00.0000      | 2          | 11  | 100.00%    | 0.00%     | 0.00%   | 100.00%     | <div></div> |

Cumulated Statistics of the Parts which the Drain Deleted

Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



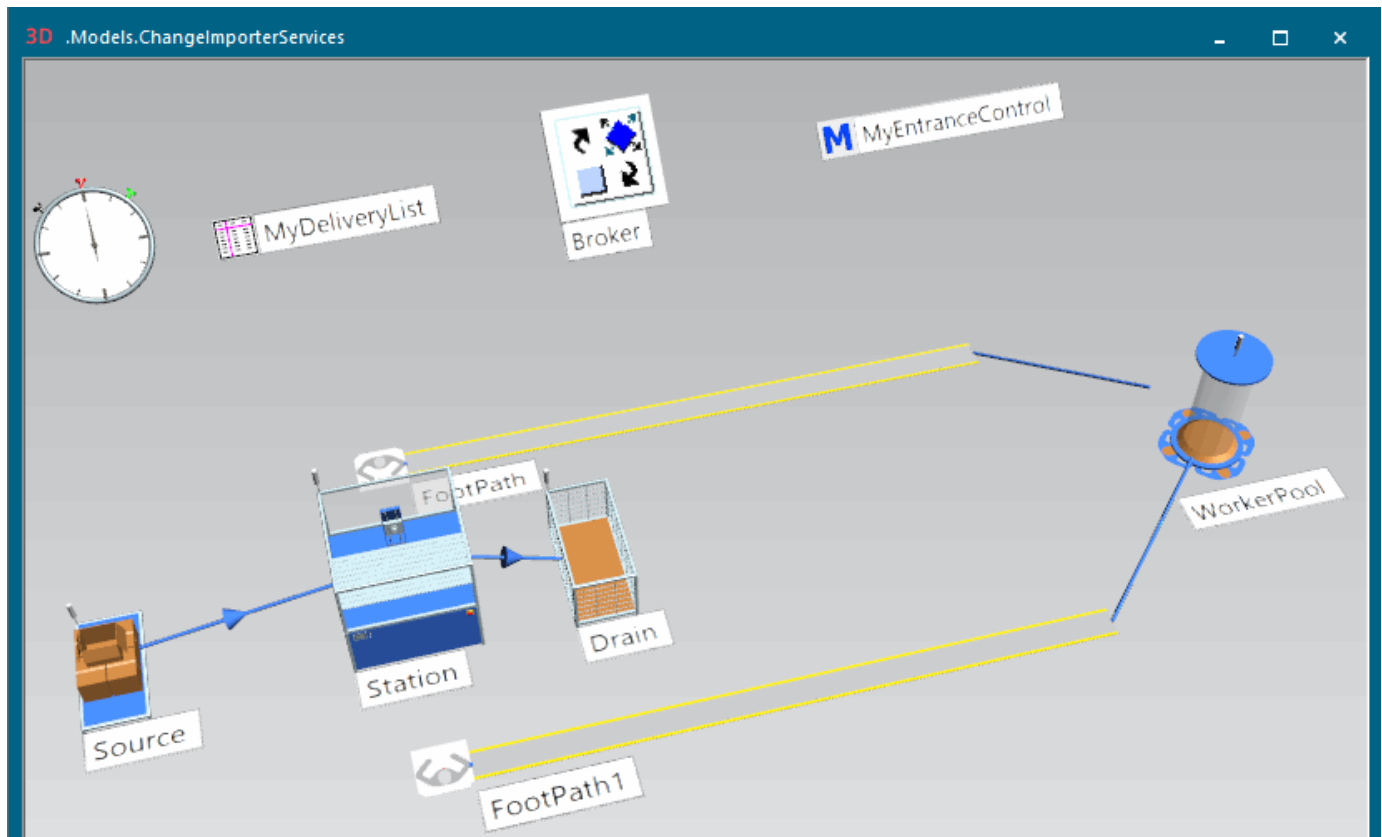
### Back to [Define Controls for Point-Oriented Objects](#)

## Change the Importer Services in the Entrance Control

You can change the **Services**, which the *importer* requests, in the **Entrance Control**.

To activate the **Entrance Control** of **point-oriented objects before** the standard actions are started, when a part enters the object, select **Before Actions**.

The finished simulation model looks like this:

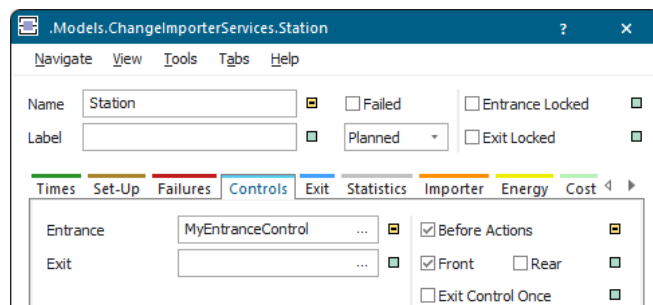


To create the model, proceed like this:

- Insert and configure the *Source* which produces the parts:

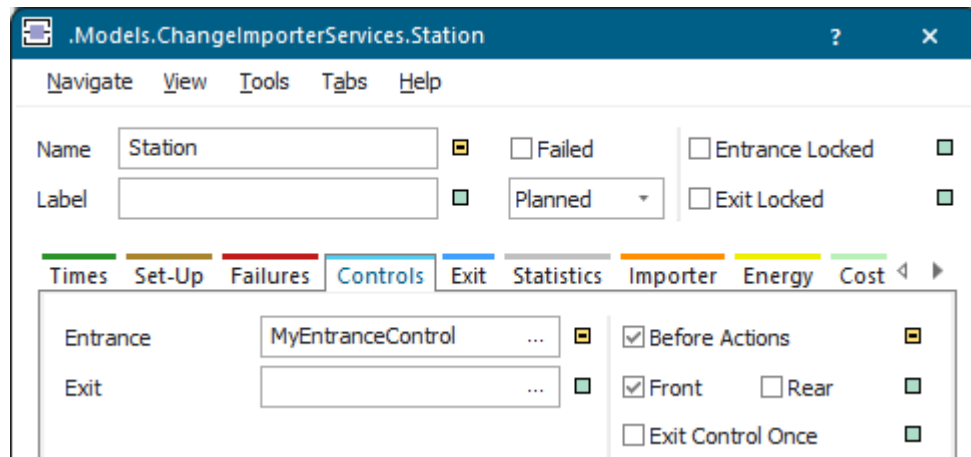
Select the **Time of Creation**. We selected **Delivery Table** and then selected the name of the **Delivery Table**, which we inserted before, namely the *DataTable* named *MyDeliveryList*. Selecting the table automatically applied the correct format.

- Enter the required data into the **Delivery Table**. We typed in the data shown in the screenshot below, which sets when the three different parts are delivered.

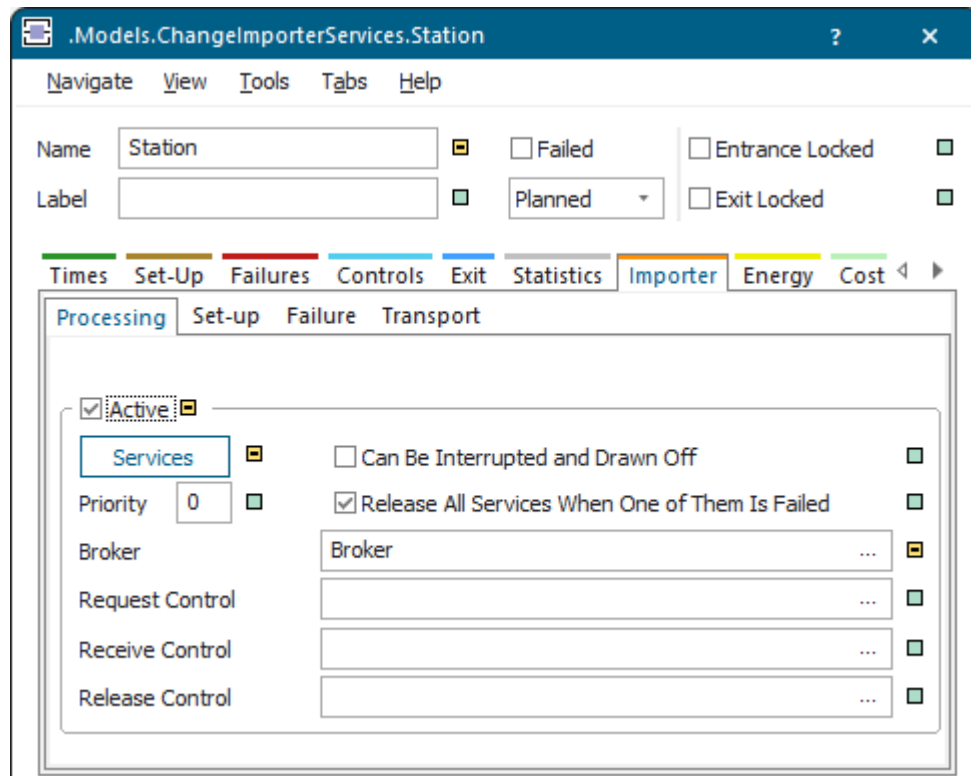


|        | time 1        | object 2  | integer 3 | string 4 | table 5    |
|--------|---------------|-----------|-----------|----------|------------|
| string | Delivery Time | MU        | Number    | Name     | Attributes |
| 1      | 1:00.0000     | .MUs.Part | 1         | PartA    |            |
| 2      | 2:00.0000     | .MUs.Part | 1         | PartB    |            |
| 3      | 3:00.0000     | .MUs.Part | 1         | PartC    |            |
| 4      |               |           |           |          |            |

- Insert and configure the station which processes the parts. We used a *Station*. We selected the method *MyEntranceControl*, which we inserted before, as the **Entrance Control** and selected **Before Actions**.



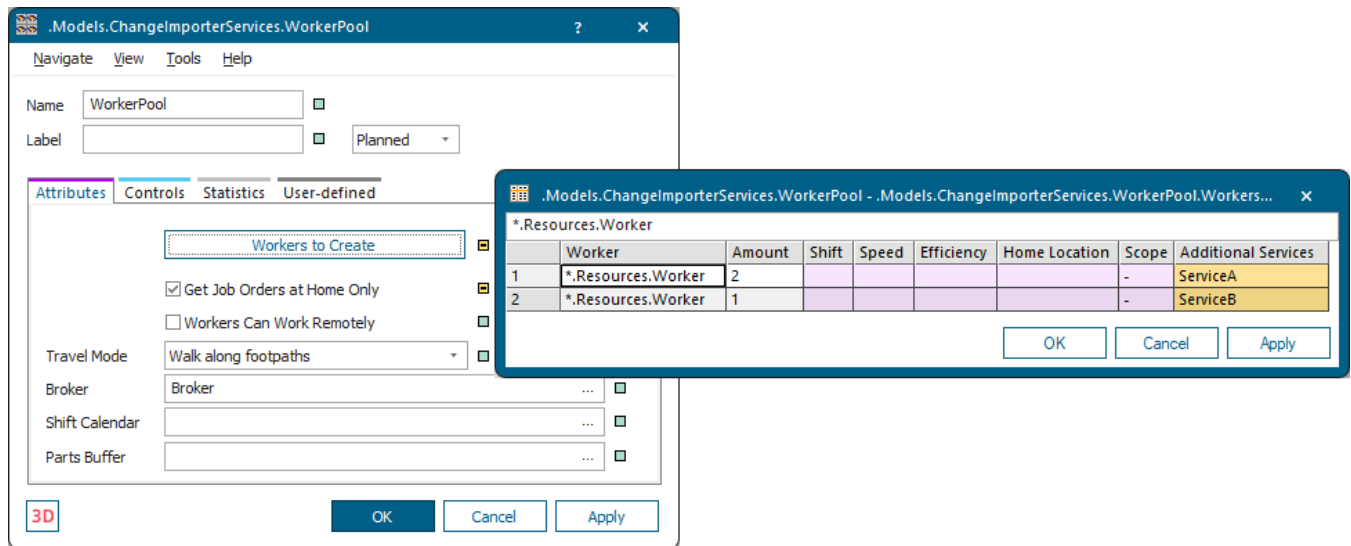
Then, activate the *processing importer* on the **Tab Importer**.



You do not need to change any of the other settings.

- Insert the *Drain*, which removes the parts from the plant.
- Insert the two *Workplaces*, which service the processing station, the *FootPaths* in which the *Workers* walk to and from the *WorkerPool*, and the *Broker*. This is the *Broker* who manages the *Workers* on the tab **Importer** of the *Station*.
- Configure the *WorkerPool*. Click **Creation Table** and enter the data shown in the screenshot below. Select the check box **Get Job Orders at Home Only**.





- Program the **Entrance Control**. We typed in the source code below. It creates the local variable `servicesTable` and gets the services table of the *processing importer* of the *Station*. It then checks the names of the parts and writes the name of the service and the amount for that part into the `servicesTable`. It finally sets the local variable `servicesTable` as the services table of the *importer* of the *Station*.

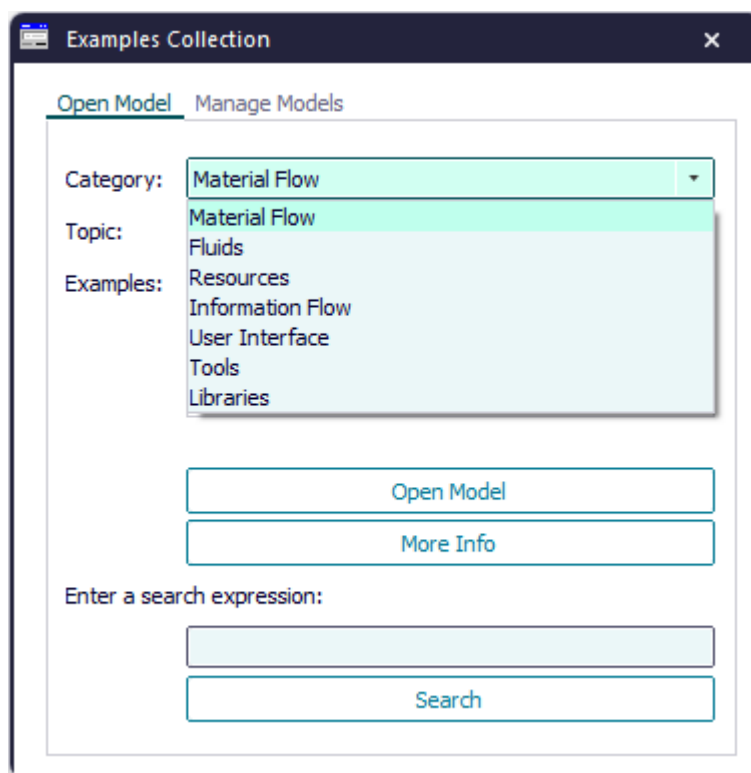
The source code looks like this:

```
var servicesTable: table[string,integer,string]

Station.imp.getServices(servicesTable)
servicesTable.delete
switch @.Name
  case "PartA"
    servicesTable.writeRow(1,1, "ServiceA",2)
  case "PartB"
    servicesTable.writeRow(1,1, "ServiceB",1)
  case "PartC"
    servicesTable.writeRow(1,1, "ServiceA",1)
    servicesTable.writeRow(1,2, "ServiceB",1)
end
Station.imp.setServices(servicesTable)
```

- When you click **Apply**, *Plant Simulation* show a message telling you that you have to activate the check box **Before Actions**, if the instructions in the control are to apply to the active part.
- Run the simulation and watch what happens. To follow how the events are processed during the simulation, you can click  in the *EventController* and then click  in the *EventDebugger*. Compare the event list with the services which the **Entrance Control** entered into services table, which the button  opens.

Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



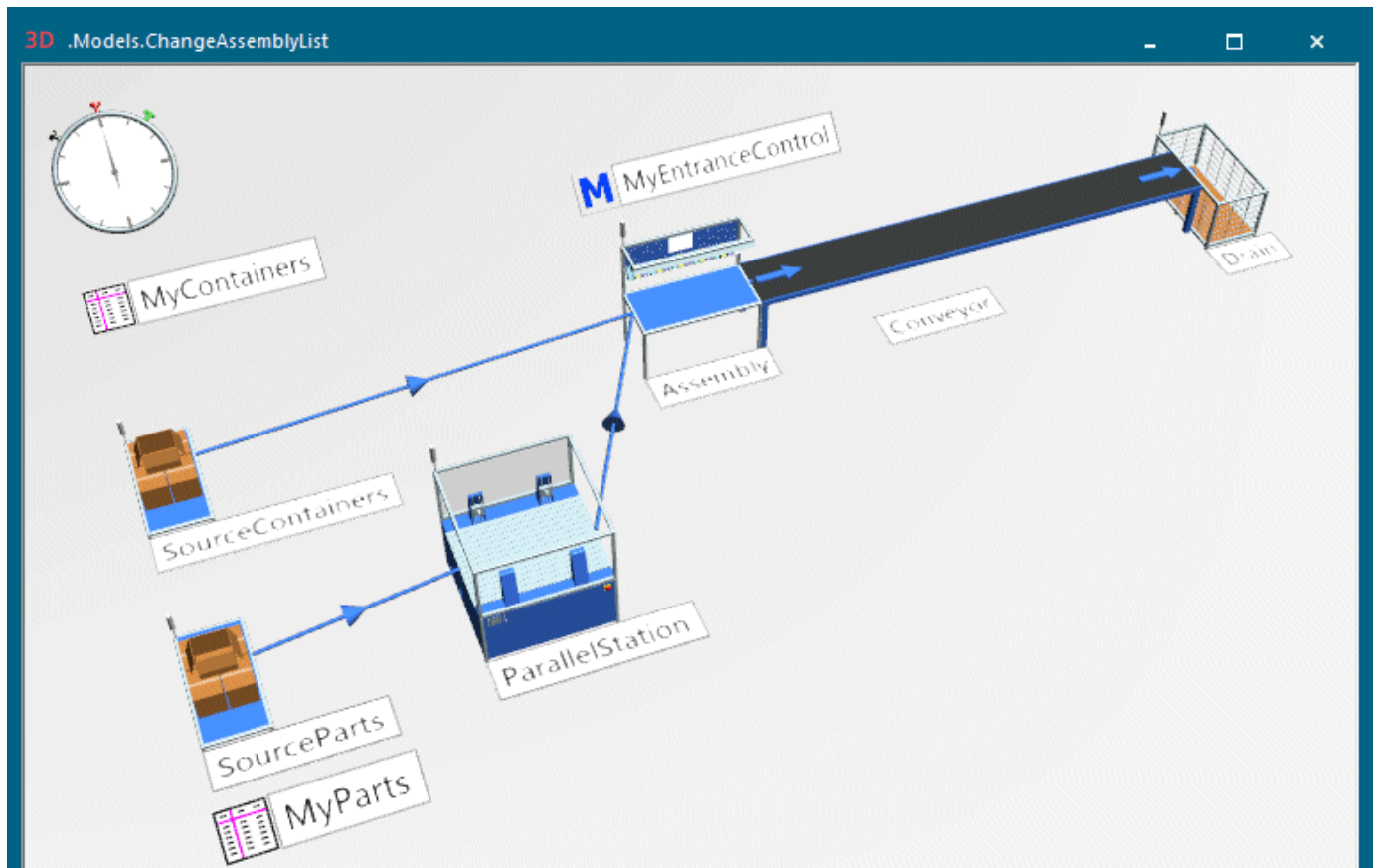
**Back to** [Define Controls for Point-Oriented Objects](#)

## Change the Assembly Table in the Entrance Control

You can change the **Assembly Table** of the *AssemblyStation* in the **Entrance Control**.

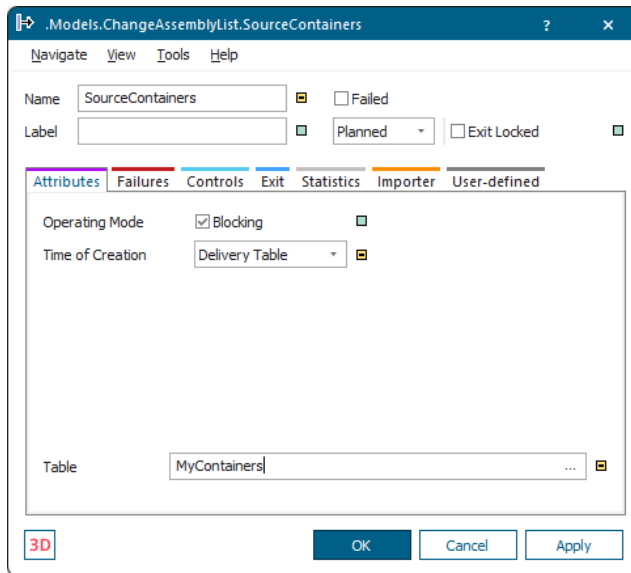
To activate the **Entrance Control** of **point-oriented objects before** the standard actions are started, when a part enters the object, select **Before Actions**.

The finished simulation model looks like this:



To create the model, proceed like this:

- Insert and configure the *Sources* which produce the parts.
  - The *SourceContainer* produces the pallets onto which the parts are loaded.  
 Select the **Time of Creation**. We selected **Delivery Table** and then selected the name of the **Delivery Table**, which we inserted before, namely the *DataTable* named *MyContainers*. Selecting the table automatically applied the correct format.
  - Enter the required data into the **Delivery Table**. We typed in the data shown in the screenshot below, which sets when the two containers are delivered.

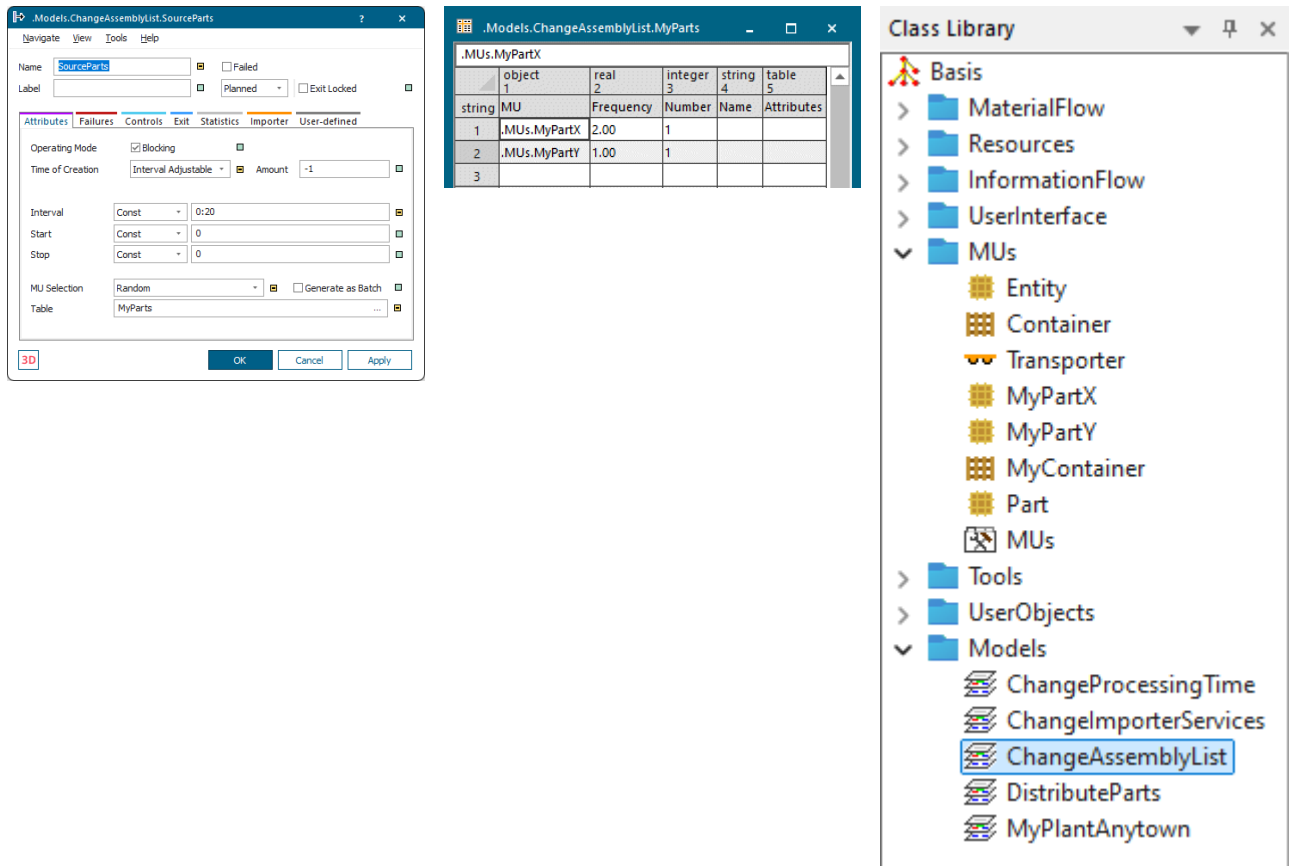


|        | time          | object           | integer | string | table      |
|--------|---------------|------------------|---------|--------|------------|
|        | 1             | 2                | 3       | 4      | 5          |
| string | Delivery Time | MU               | Number  | Name   | Attributes |
| 1      | 1:00.0000     | .MUs.MyContainer | 1       | A      |            |
| 2      | 2:00.0000     | .MUs.MyContainer | 1       | B      |            |
| 3      |               |                  |         |        |            |

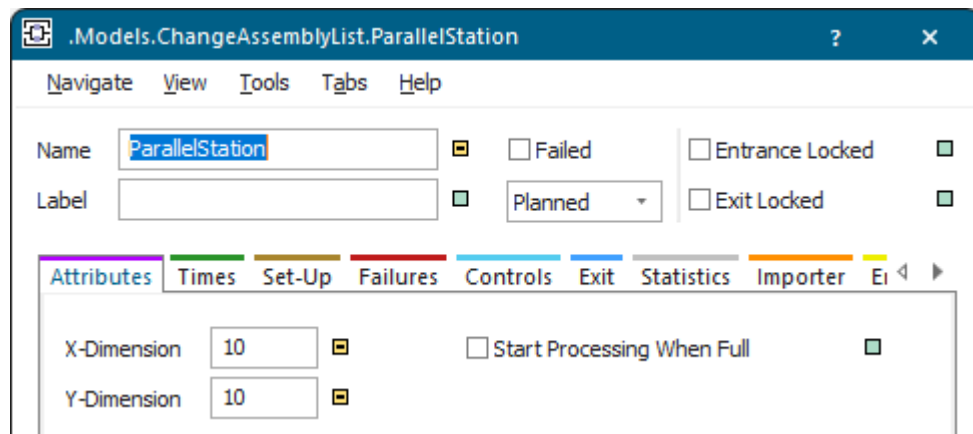
- The *SourceParts* produces the parts which are loaded onto the pallet.

Enter the **Interval**. We entered an interval of 20 seconds, i.e., 0 : 20.

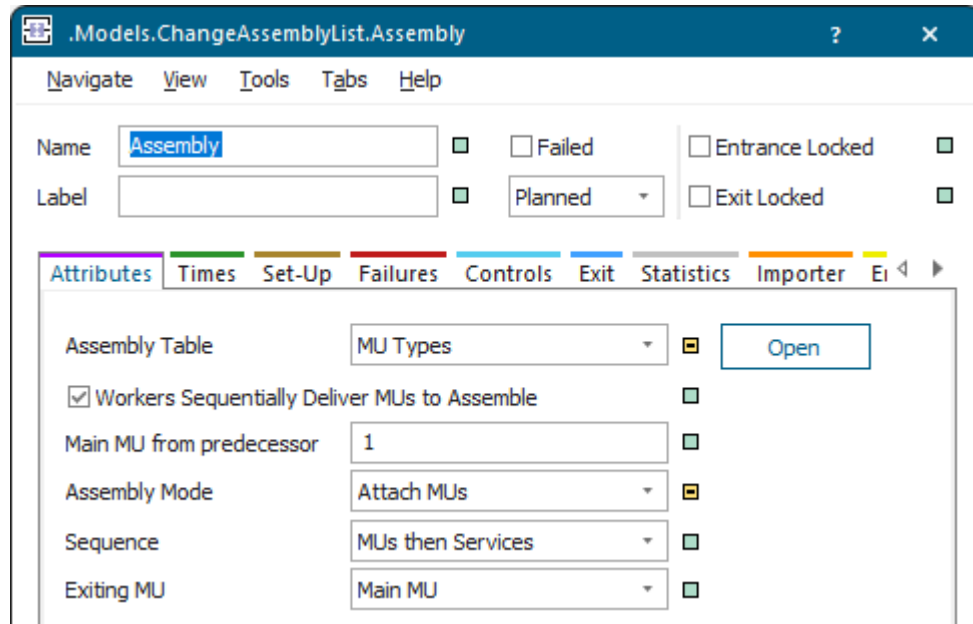
Select the **MU Selection**. We selected **Random** and then the name of the **table**, which we inserted before, namely the *DataTable* named *MyParts*. Selecting the table automatically applied the correct format. We typed in the data shown in the screenshot below, which sets the frequencies at which the two parts are created.



- Insert and configure the station which processes the parts. We used a *ParallelStation* and increased its capacity to 100 parts, i.e., we entered an **X-Dimension** and a **Y-Dimension** of 10 each. We did not change any of the other settings.

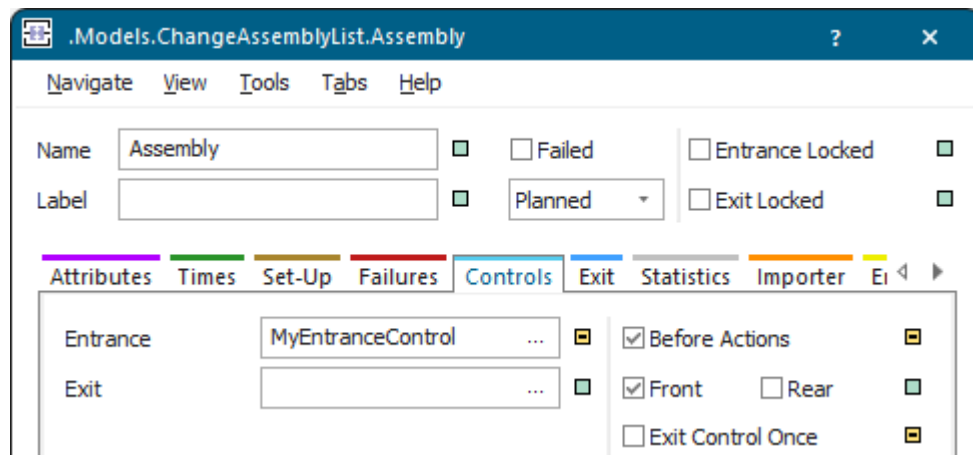


- Insert and configure the **AssemblyStation** which assembles the parts that arrive from the *Sources*. We assemble the parts according to the **Assembly Table**. As we want to attach the parts, we selected the **Assembly Mode > Attach MUs**.



- Select and program the **Entrance Control**. We selected the method *MyEntranceControl*, which we inserted before, as the **Entrance Control** and selected **Before Actions**.

We typed in the source code below. It creates the local variable *assList*. It then checks the names of the containers, A or B, and writes the name and the amount for that part, *MyPartX* or *MyPartY*, into the *assList*. It finally sets the local variable *assList* as the assembly list of the *AssemblyStation*.



The source code looks like this:

```
var assList: table[string,integer]

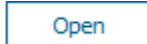
assList.create
switch @.Name
case "A"
    assList.writeRow(1,1, "MyPartX",1)
    assList.writeRow(1,2, "MyPartY",2)
case "B"
    assList.writeRow(1,1, "MyPartX",2)
```

```

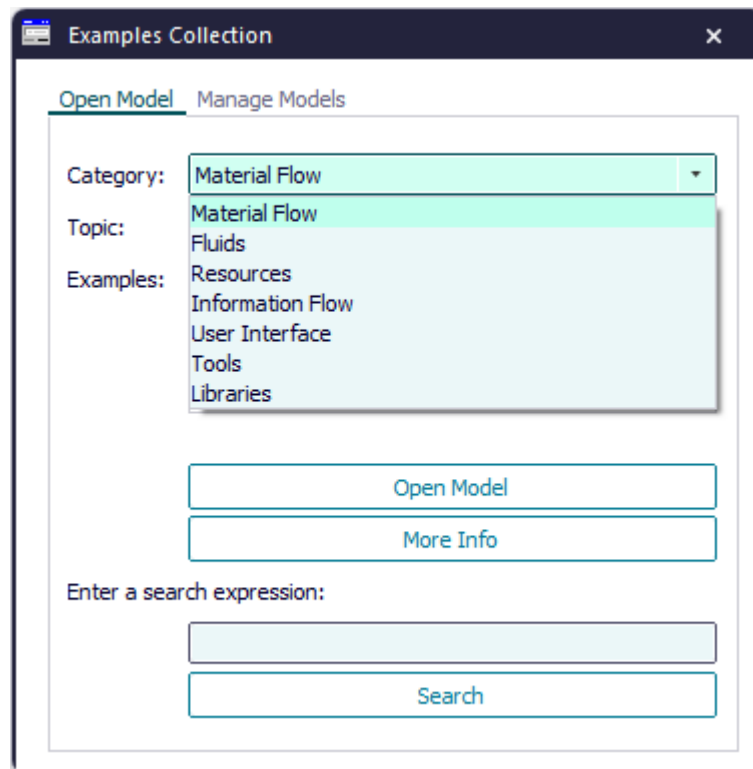
    assyList.writeRow(1,2, "MyPartY",1)
end
Assembly.AssemblyTable := assyList

```

- Insert the *Conveyor*, which transports the assembled parts, and the *Drain*, which removes them from the plant.
- Run the simulation and watch what happens. To follow how the events are processed during the simulation, you can click  in the *EventController* and then click  in the *EventDebugger*. Compare the event list with the parts, which the **Entrance Control** entered into the assembly table.

Click **Assembly Table**   to open the **Assembly Table**.

Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



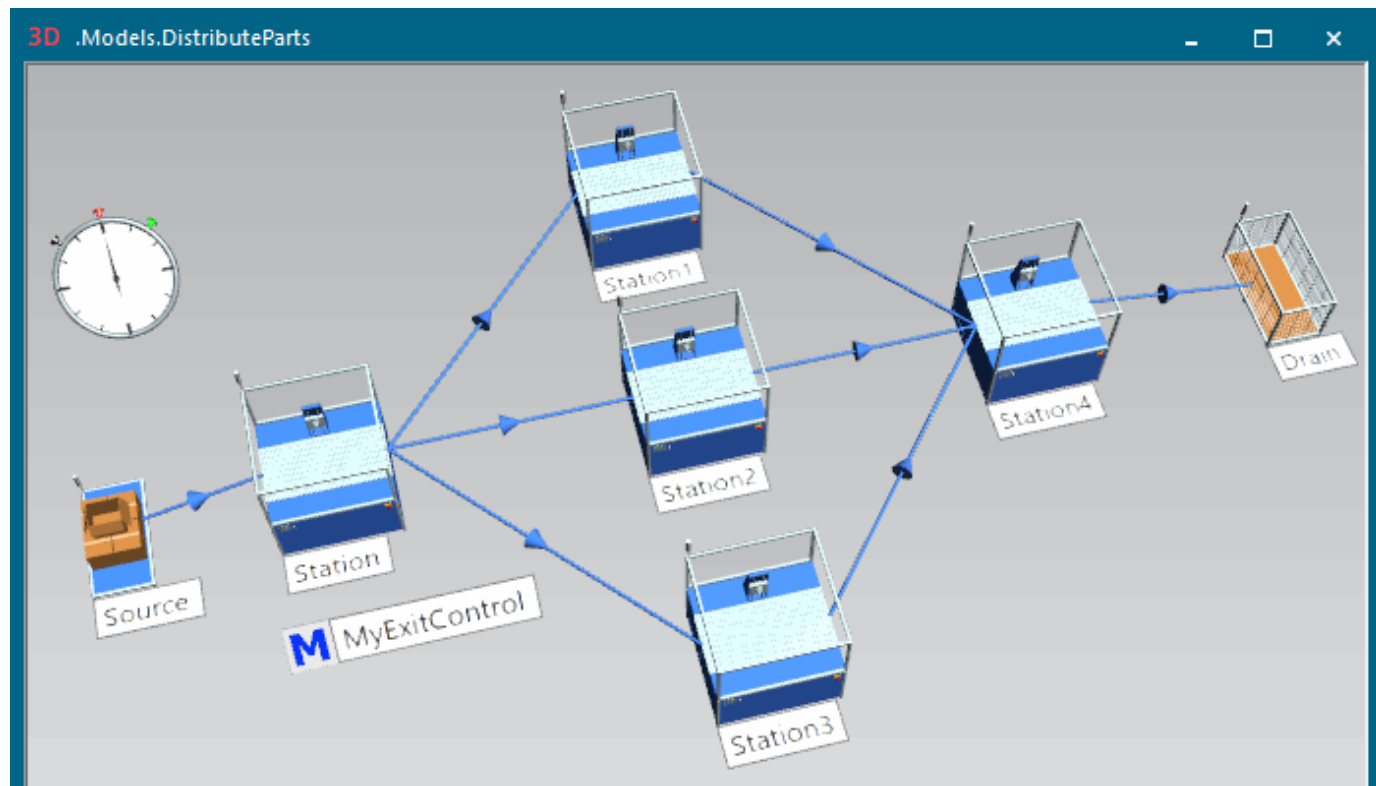
Back to [Define Controls for Point-Oriented Objects](#)

## Distribute Parts with an Exit Control

You can determine how a station moves the parts, which it has processed, on to its successors in the **Exit Control**.

In our example the *Source* produces 1000 parts and moves these on to the *Station*. We are going to count the number of incoming parts in the **Exit Control**, and then, after a certain number has been reached, move the following parts on to a different successor.

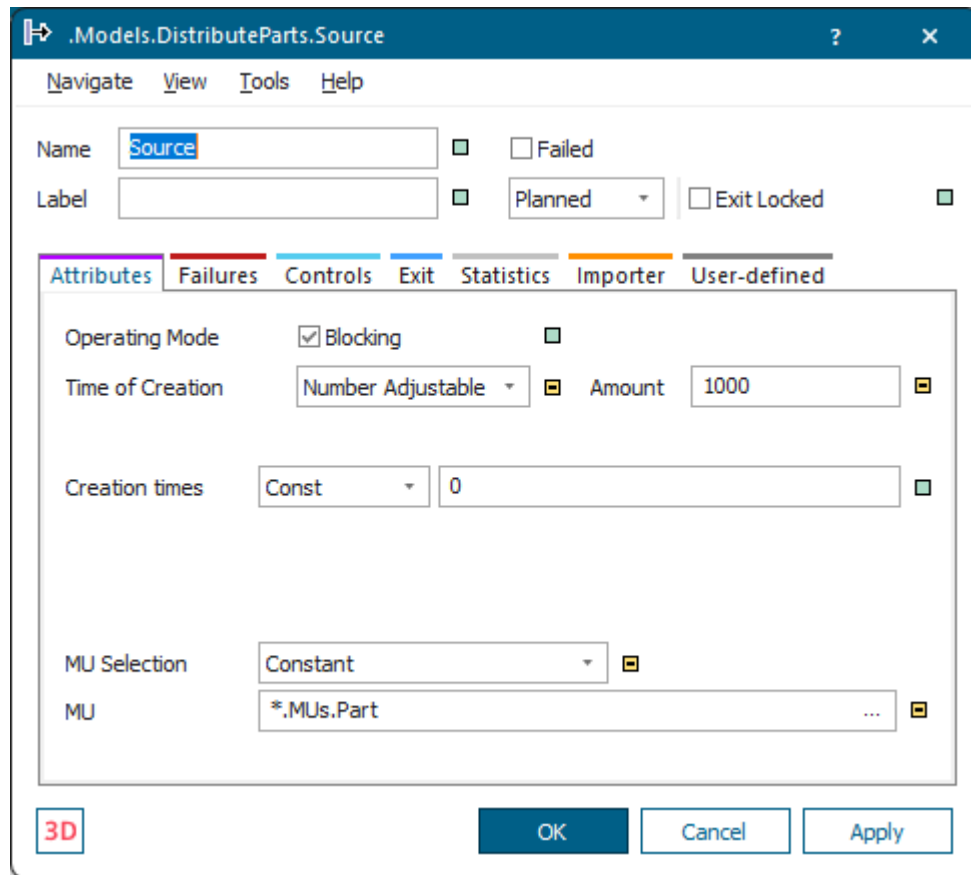
The finished simulation model looks like this:



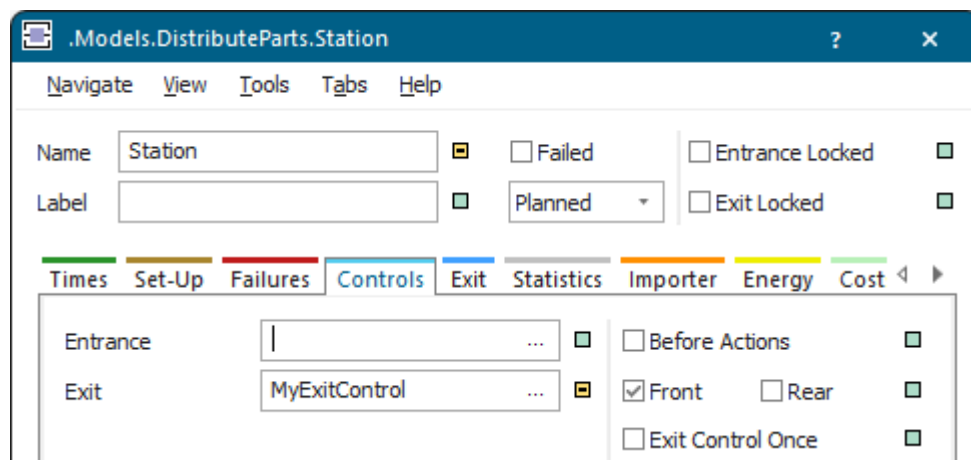
To create the model, proceed like this:

- Insert and configure the *Source* which produces the parts. We selected and entered the settings below and did not change any other settings.





- Insert and configure the stations which process the parts. We inserted four *Stations*. We use them with their default settings, except for the *Station* that distributes the parts across its successors.
- Select and program the **Exit Control** for this station.



- We selected the Method named *MyExitControl* and entered the source code below.

```
var n: integer
n := Station.StatNumIn
if n <= 100
```

```

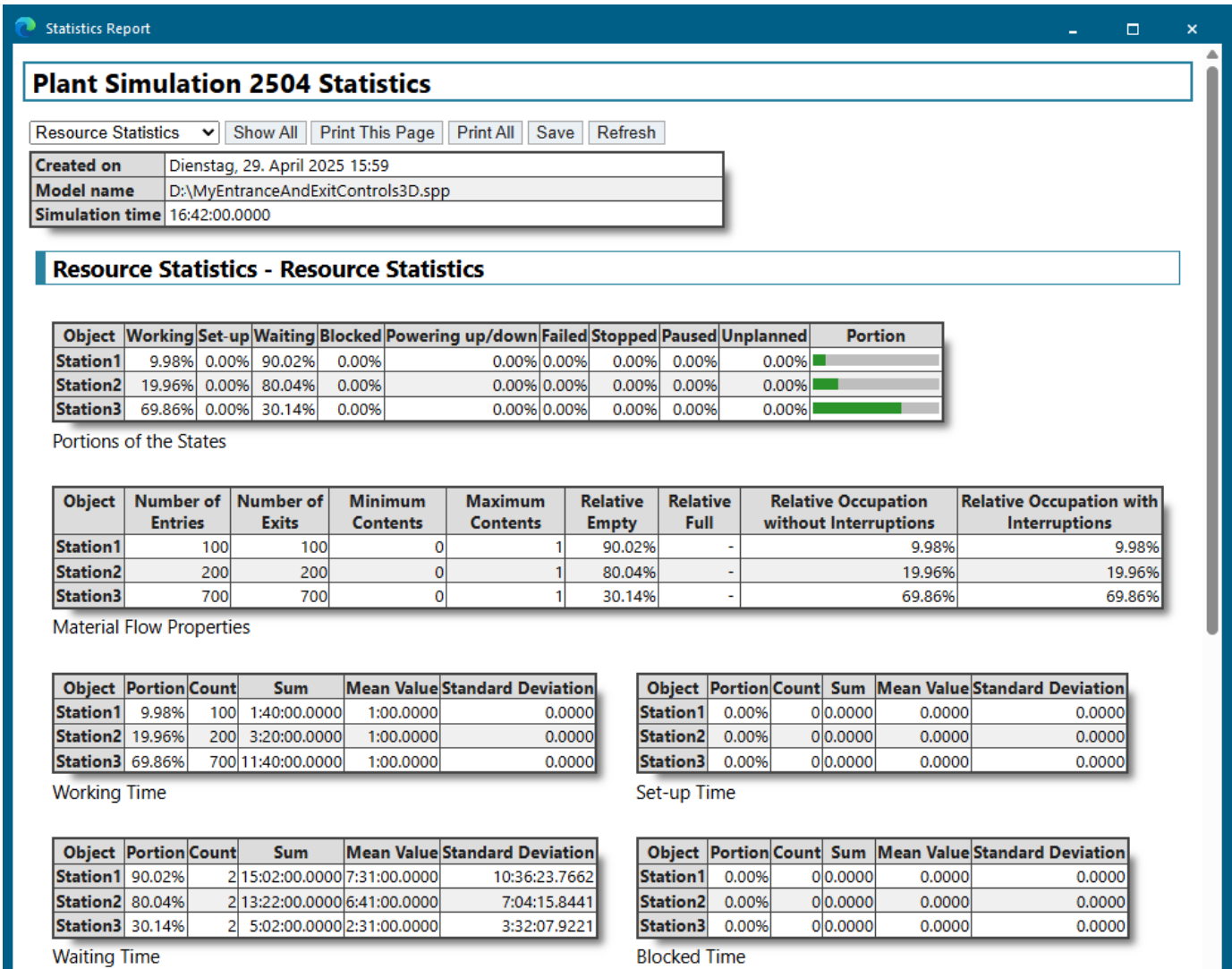
    @.move(Station1)
elseif n <= 300
    @.move(Station2)
else
    @.move(Station3)
end

```

We declared the *Variable* named *n* and the data type *integer*. We typed in *n := Station.StatNumIn*. This reads the statistics value *StatNumIn* of the *Station*. We then pressed **Enter** again, changed to the **Template** menu and selected **if... elseif... end**. This inserted an **if... elseif... end**-statement into the source code and saved us from typing it in. It also showed us, with the expressions highlighted in red, which parts of the source code we have to replace with our own instructions.

- First, we replaced the **condition** with our condition, namely *n* less than or equal to 100, i.e., we typed in *n <= 100*.
- Next, we replaced the **then\_branch** with *@.move(Station1)*. This instruction tells *Plant Simulation* to move the first 100 parts to *Station1*.
- Then, we replaced the **elseif condition** with our condition, *n* less than or equal to 300, i.e., we typed in *n <= 300*. We replaced the remaining **then\_branch** with *@.move(Station2)*. This moves the next 200 parts to the station *Station2*.
- We then pressed **Enter**, typed in **else** and pressed **Enter** again.
- Finally, we typed in *@.move(Station3)*. This moves the remaining 700 parts to *Station3*. The **@** sign in our source code identifies the part, which is going to be moved.

Run the simulation and check the *statistics report*. It shows that the parts are distributed as we programmed them to do: The first **100** parts moved to *Station1*, the next **200** to *Station2*, and the remaining **700** to *Station3*.



## Compare Define Controls for Length-Oriented Objects

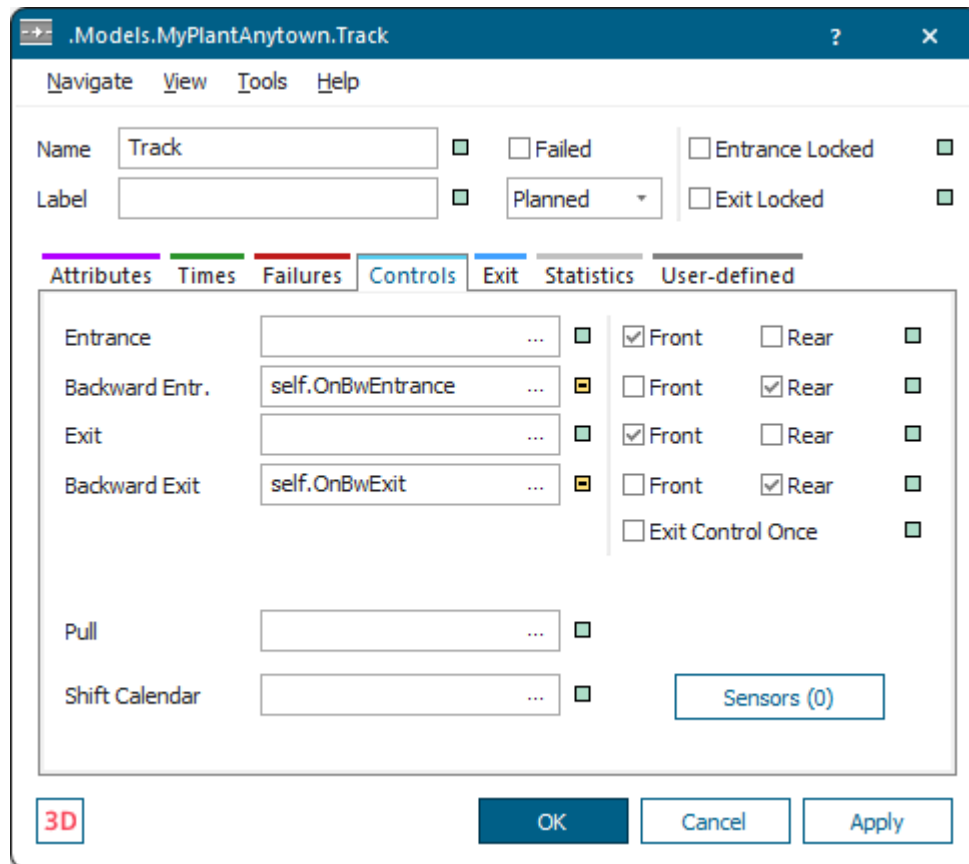
### Back to Create Entrance and Exit Controls

### Define Controls for Length-Oriented Objects

You can define controls for length-oriented objects.

The **length-oriented objects** [Track](#) , [TwoLaneTrack](#) , [Conveyor](#) , [Turnplate](#) , [Turntable](#) , [AngularConverter](#) , and [Converter](#)  take their own length and the length of the *MUs*, which move on them, into account during the simulation runs.

In addition to the forward **Entrance Control** and the forward **Exit Control**, they provide a **Backward Entrance Control** and a **Backward Exit Control**. *Plant Simulation* activates these controls, when the **Transporter**  drives in reverse on them.

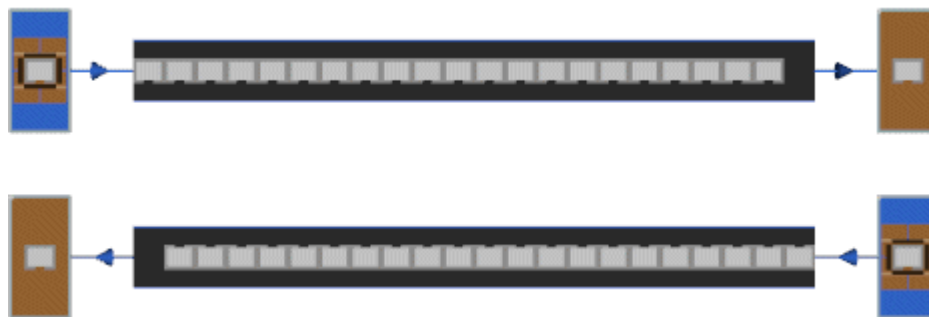


### Note

The *Transporter* backs up on the *Track*, it does not turn around. This means that it drives in reverse with its front end still pointing in the direction of the flow of materials instead of its front end pointing in the opposite direction!

### Note

The **Front** of the part always moves toward the end of the length-oriented object into the direction with which you inserted it along the direction of motion of the material flow. When you insert a length-oriented object, for example a *Conveyor*, from left to right, the **Front** of the part is located on the right-hand side and moves toward the right. When you insert a length-oriented object from right to left, the **Front** of the part is located on the left-hand side and moves toward the left.



This also applies to parts which move backward when you select the check box **Backward** of the *Conveyor* or the *Transporter*! The **Front** of the part keeps on pointing toward the end of the length-oriented object into the direction with which you inserted it along the direction of motion of the material flow.

*Plant Simulation* activates the **Entrance Control** you entered:

- When the **Front** or the **Rear** of the *MU* has entered the **length-oriented object**.

Remember that a *MU*, for which you entered a length, does not move onto these objects all at once, but continually. Depending on the length and the speed of the *MU*, there will a delay between the time the front and the time the rear of the *MU* enters the object.

- Select **Front** to activate the control as soon as the front of the *MU* has entered, i.e., is located on the object. This means that changing the processing time in the entrance control does not affect the *MU* that already entered the object. To set the processing time depending on the *MU* that arrives, define the processing time in a formula.
- Select **Rear** to activate the control as soon as the rear of the *MU* has entered the object.

*Plant Simulation* activates the **Backward Entrance Controls**:

When the **Front** or the **Rear** of the *Transporter* has moved onto the **length-oriented objects** *Track*, *TwoLaneTrack*, or *Conveyor* at their entrance while it backs up.

Remember that a *Transporter*, for which you entered a length, does not move onto these objects all at once while backing up, but continually. Depending on the length and the speed of the *Transporter*, there will a delay between the time the back and the time its front is located on the object:

- Select **Rear** to activate the control as soon as the rear of the *Transporter* has moved onto the object while driving in reverse.
- Select **Front** to activate the control as soon as the front of the *Transporter* has moved onto the object driving moving in reverse.
- You can also select both check boxes, **Front** and **Rear**, for the entrance control and the exit control. *Plant Simulation* activates the control as soon as the **Front** or the **Rear** of the *Transporter* enter or exit the object.

*Plant Simulation* activates the **Backward Exit Controls**:

When the **Front** or the **Rear** of the *Transporter* has moved onto the **length-oriented objects** at their exit while it drives in reverse.

The check boxes **Front** and **Rear** set, when the *Transporter* triggers the *Methods*.

- Select **Rear** to activate the control as soon as the back of the *Transporter* has moved onto the object at its exit while driving in reverse.

#### Note

*Plant Simulation* calls the rear-activated exit control once only. It does not override the default exit strategy.

- Select **Front** to activate the control as soon as the front of the *Transporter* has moved onto the object at its exit while driving in reverse. The exit control then has to move the part to another object, as the built-in behavior of the e, i.e., moving the *MU* on to the successors along the *Connectors*, has been overridden by the exit control.

#### Note

The same *MU* can call the **front-triggered Exit Control** more than once, if that *MU* could not exit the object and entered itself into the **Blocking List** of the destination object. As soon as the destination object is ready to receive the *MU*, the *MU* is assigned a new **Out** event, which triggers the **front-triggered Exit Control** again.

- You can also select both check boxes, **Front** and **Rear**, for the entrance control and the exit control. *Plant Simulation* activates the control as soon as the **Front** or the **Rear** of the *MU* enter or exit the object.

Compare [Define Controls for Point-Oriented Objects](#)

Compare [Model Transport Systems](#)

Back to [Create Entrance and Exit Controls](#)

## Create a Sensor [material flow objects]

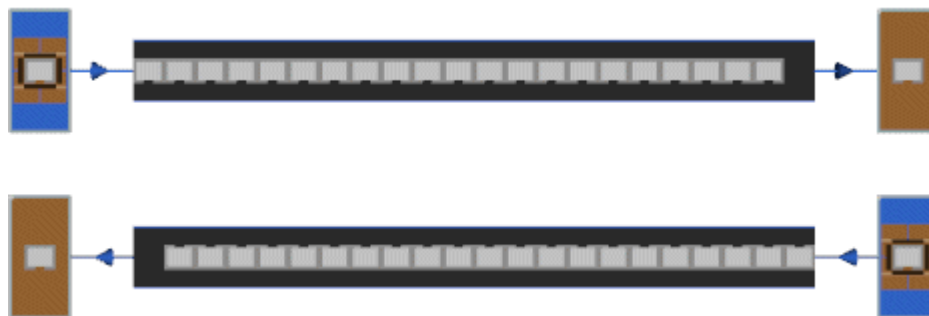
The material flow objects have built-in sensors, i.e., the entrance controls and the exit controls that the arriving or the leaving *MUs* trigger. A triggered sensor calls the *Method* object, whose name you specified as the **Entrance Control** or as the **Exit Control**. These user-defined controls will then replace the built-in properties of the object.

For the length-oriented objects *Track* , *TwoLaneTrack* , *Turntable* , and *Conveyor* , for the *Transporter* , and for the *Tank*  you yourself can, in addition to the entrance controls and to the exit controls, define sensor controls anywhere on the object. The *Track/TwoLaneTrack* and the *Conveyor* activate the *Methods* you assigned to the sensor, once a *MU* passes the sensor, which in your plant might be a light barrier, or a similar device. In the *Method* assigned to the control you can program the action you want *Plant Simulation* to take. You might, for example define the conditions that apply when the parts are transferred onto the next material flow object, or you might change the target velocity, and the icon, of a *Transporter*, when its front reaches the sensor.

#### Note

The **Front** of the part always moves toward the end of the length-oriented object into the direction with which you inserted it along the direction of motion of the material flow. When you insert a length-oriented object, for example a *Conveyor*, from left to right, the **Front** of the part is located on the

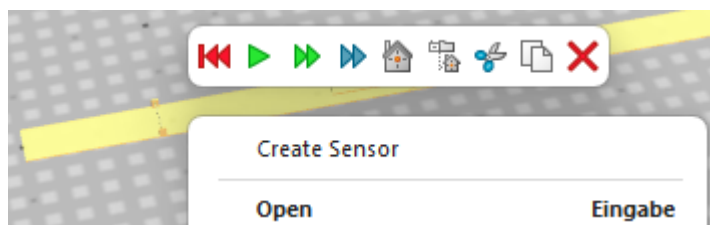
right-hand side and moves toward the right. When you insert a length-oriented object from right to left, the **Front** of the part is located on the left-hand side and moves toward the left.



This also applies to parts which move backward when you select the check box **Backward** of the *Conveyor* or the *Transporter*! The **Front** of the part keeps on pointing toward the end of the length-oriented object into the direction with which you inserted it along the direction of motion of the material flow.

To create **sensors** in the dialog of the length-oriented objects:

- Click the position at which you would like to insert the sensor with the right mouse button and select **Create Sensor** on the context menu. Specify the data defining to the sensor into the dialog that opens.

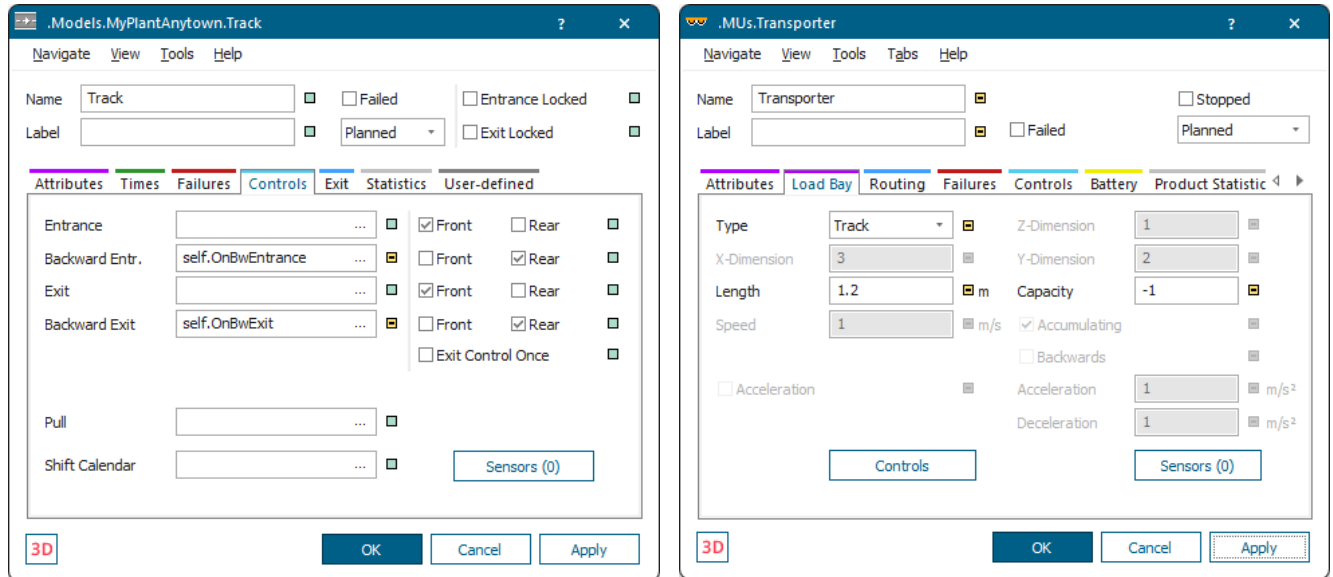


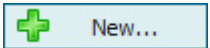
- Or click the tab **Controls**.

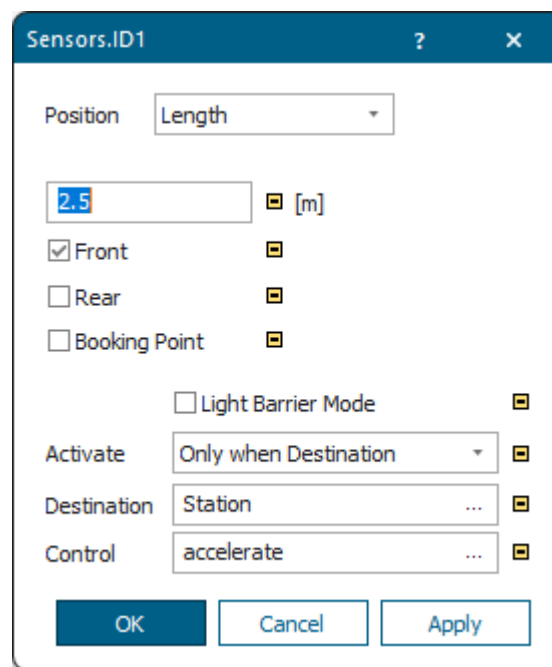
#### Note

The *Transporter* shows the button **Sensors** on the tab **Load Bay**, when you select **Track** or **Line**. Click **Apply** to activate the button.

- Click **...** and select the name of the *control method* in which you programmed the action you would like the *Transporter* to take. In our example, we want it to turn around, when its front passes the sensor.



- To insert additional sensors, click **Sensors**.
- To create a new sensor and open the dialog **Sensor**, click **New** .



- The **ID** in the title bar of the dialog is the number that *Plant Simulation* automatically assigns to the sensor, when it creates it. You can use this unique identifier to access that sensor from within *Methods*.
- Select the type of position, **Relative** or **Length**, from the drop-down list and specify the position of the sensor.
  - ◊ For **Relative** you can specify a value between 0 and 1, and *Plant Simulation* show **0..1** to the right of the text box.



- ♦ For **Length** you can specify a value between 0 and the length of the object.

*Plant Simulation* uses the length unit you selected under **File > Model Settings/Preferences > Units > Length**. When you specify an invalid value, *Plant Simulation* changes the color of the text box to red.

- When you define a sensor, you can select when a *MU* triggers the sensor:

**Always**, independent of the destination of the *MU*.

Or **Only when** the *MU* has the same **destination** as the one you entered into the sensor.

**Automatic Routing** of the *Transporter* and **Automatic Routing** of *Part* and *Container* also includes this destination object. When you assign a **Destination** to the *MU* while automatic routing is active, it drives to the sensor. Once there it activates the **Sensor Control**. For the *Transporter* it also activates the **Destination Control**. The same destination object can be defined in several sensors. It may also be directly reached via different *routes*. Even when a direct route leads to the destination object, the *MU* drives to the nearest sensor, when this sensor can be reached on a shorter route than the destination object itself.

- Specify the name of the object to which the *Transporter* drives into the text box **Destination**.
- Click  and select the name of the *Method* **M**, which the sensor calls, in the dialog **Select Object**. To open the dialog of the object, whose name you entered into the text box, click in the text box, and press **F2**.

The source code of our *Method* named *accelerate* reads like this:

```
@.Speed := 50
@.currIcon := "car_fast"
```

As soon as the sensor calls this *Method*, it passes the **Sensor ID** as parameter. When the *Method* expects an *integer* parameter, the sensor passes the **Sensor ID** to the *Method*. If you do not specify an *integer* parameter, the *Method* will be called without a parameter.

Or you can right-click in the text box. Then, select **Create Control** on the context menu to create a *control method*, which is a user-defined attribute of data type *method* of this object:

- ♦ Type a meaningful name into the text box and select **Create Control**. *Plant Simulation* then inserts `self.Name_you_typed_in_for_the_control`, such as `self.MySensorControl`.
- ♦ Select **Create Control**. *Plant Simulation* then inserts `self.OnBuilt_in_name_of_the_control`, such as `self.OnSelect`.
- ♦ To delete this control, delete the *user-defined attribute*; deleting the name from the text box just deletes the name, but keeps the *user-defined attribute*.

Type the source code of this control into the *Method* that opens.

- Select **Front**, so that the front of the *MU* activates the *Method*. You can also select **Front** and **Rear**. Then both the front and the rear of the *MU* activate the *Method*.
- Select **Rear**, so that the rear of the *MU* activates the *Method*. You can also select **Front** and **Rear**. Then both the front and the rear of the *MU* activate the *Method*.

- Select **Booking Point**, so that the **Booking Point Length** of the *MU* calls the *Method*.
- Click **OK** to apply your settings and to close the dialog.

| ID | Position | Front | Rear | Booking point | Light barrier | Path                  | Created here |
|----|----------|-------|------|---------------|---------------|-----------------------|--------------|
| 1  | 2.5m     | x     |      |               |               | Station => accelerate | *            |
| 2  | 5m       | x     |      |               |               | self.OnSensor1        | *            |
| 3  | 8.3m     | x     |      |               |               |                       | *            |

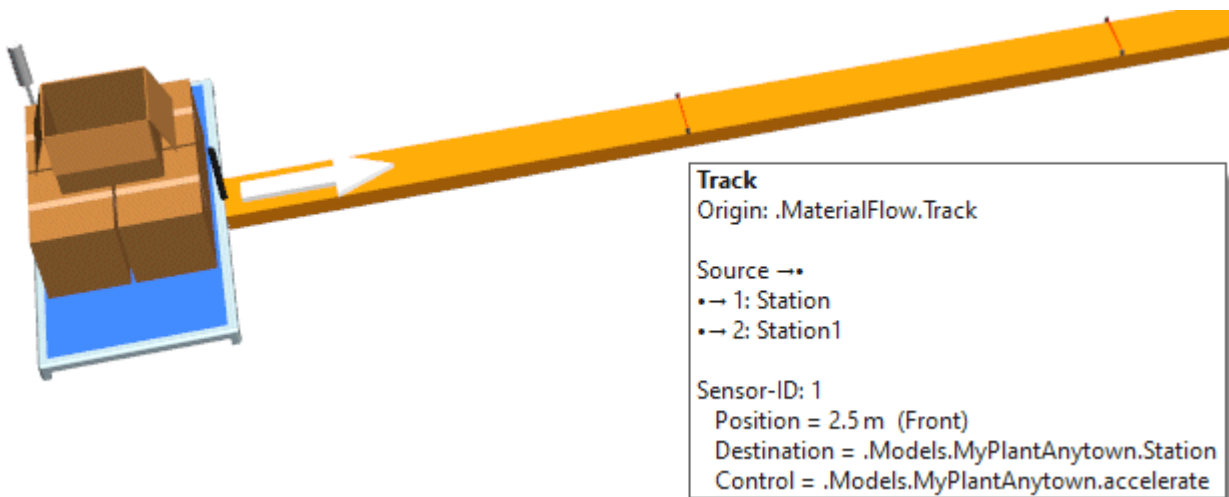
- When you click the **Close** button, *Plant Simulation* updates and shows the **number of sensors** on the button **Sensors (1)** in the dialog.
- To **edit** the sensor, which you selected in the list, click **Edit**. Or double-click the row that defines the sensor in the list.
- To **delete** the sensor, which you selected in the list, click **Delete**.
- To open the sensor that *Plant Simulation* inserts as a red line into the icon of the object, double-click the red line and change the settings in the dialog that opens. Or click the sensor with the right mouse button and select **Open Sensor** in the context menu.



#### Note

You might have to zoom in to see the red line symbolizing the sensor position.

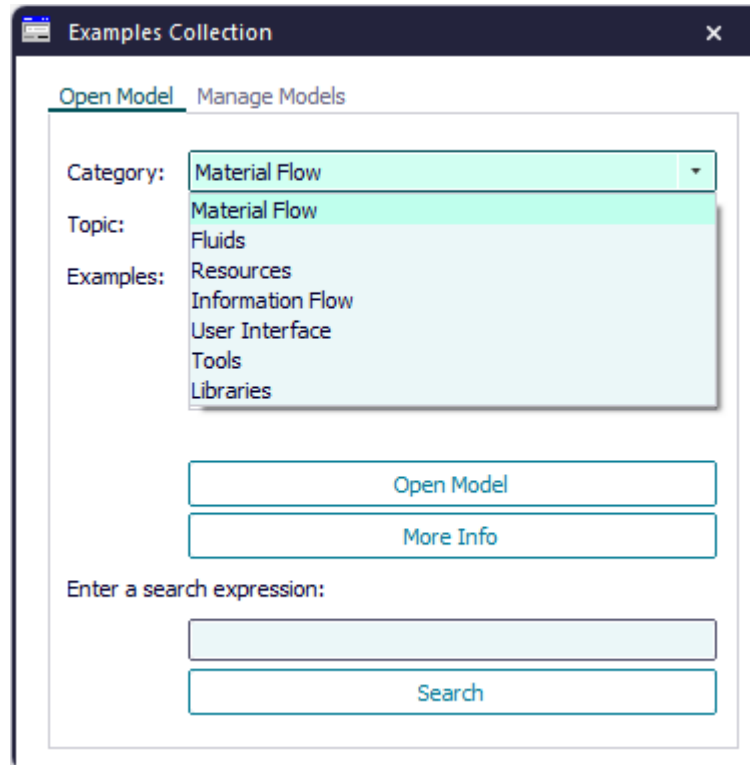
- To show a *tooltip* with information about the sensor, drag the mouse over the red line designating it on the object.



You will make extensive use of sensors when [Model Transport Systems](#). We used them in our sample applications for loading and unloading a [tugger train](#).

The fluid object *Tank* also provides sensors, compare [Configure the Tanks Which Buffer the Materials](#) and [Configure the Tanks at the End of the Plant](#).

Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



Compare [Transfer Parts from Station to Station](#)

Compare [Create Entrance and Exit Controls](#)

Compare [Customize the Behavior of Objects](#)

Back to [Model the Flow of Materials, Advanced](#)

## Create and Delete an Observer

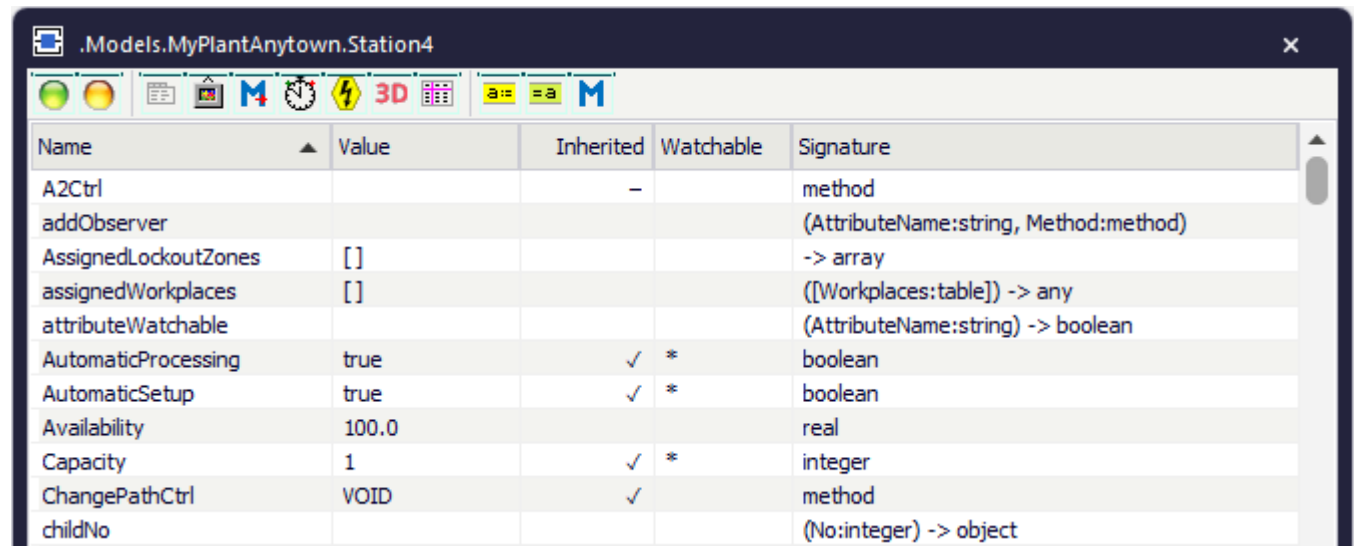
When you create your simulation model, it will often become necessary to trigger certain actions, when the watchable values of *attributes*, *read-only attributes*, or *methods* of objects change.

To accomplish this, you can create [observers](#) for most of the built-in objects.

This observer watches the value of an *attribute/read-only attribute/method* and then executes one or several *methods* when the value changes. This **method** can either be a *Method* object, which you insert into a *Frame*, or a *user-defined attribute of data type method*.

### Note

This only works, if the *attribute/read-only attribute/method* of the object is **watchable**. The column **Watchable** in the window **Show Attributes and Methods** shows this.



| Name                 | Value | Inherited | Watchable | Signature                             |
|----------------------|-------|-----------|-----------|---------------------------------------|
| A2Ctrl               |       | —         |           | method                                |
| addObserver          |       |           |           | (AttributeName:string, Method:method) |
| AssignedLockoutZones | []    |           |           | -> array                              |
| assignedWorkplaces   | []    |           |           | ([Workplaces:table]) -> any           |
| attributeWatchable   |       |           |           | (AttributeName:string) -> boolean     |
| AutomaticProcessing  | true  | ✓         | *         | boolean                               |
| AutomaticSetup       | true  | ✓         | *         | boolean                               |
| Availability         | 100.0 |           |           | real                                  |
| Capacity             | 1     | ✓         | *         | integer                               |
| ChangePathCtrl       | VOID  | ✓         |           | method                                |
| childNo              |       |           |           | (No:integer) -> object                |

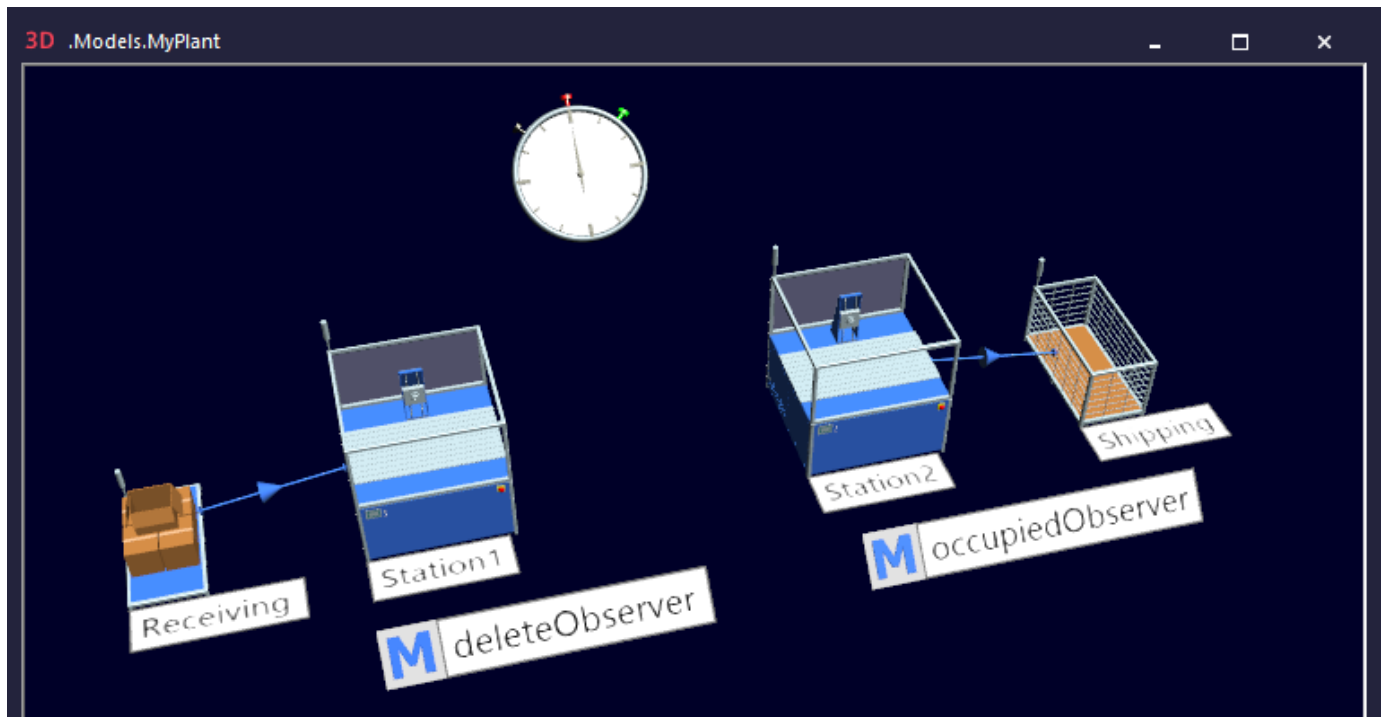
### Note

As a rule *observer methods* are called after all other controls. If you want to react to state changes before other controls, you have to use the **stopuntil-instruction** or the **waituntil-instruction** instead of an *observer*.

You can use observers for a wide range of applications, for example:

- Watch the *read-only attribute* named *NumMUs* when a certain number of *MUs* is located on the object.
- Watch the *read-only attribute* named *Empty* to check if an object is **empty** or not.

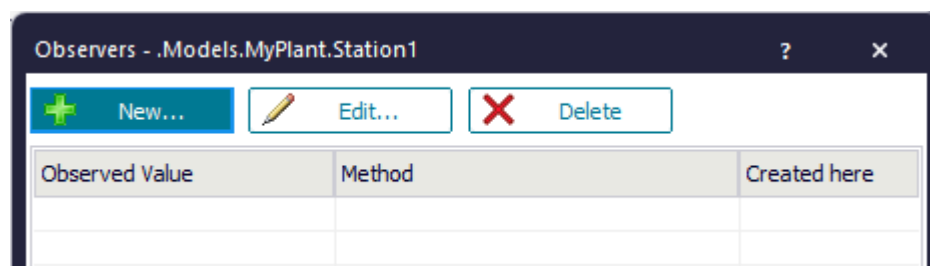
In our example, we show how to move the contents of *Station1* to *Station2*, when the value of the *read-only attribute* named *Occupied* of *Station1* changes.



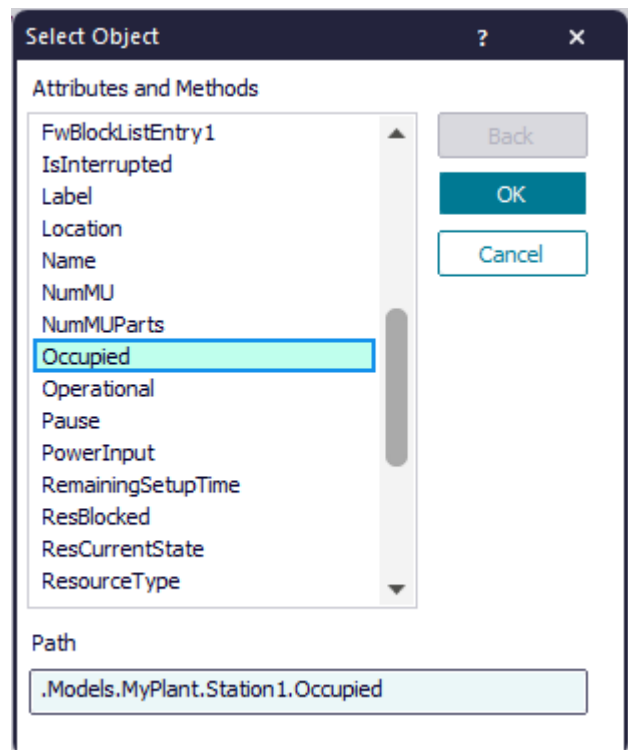
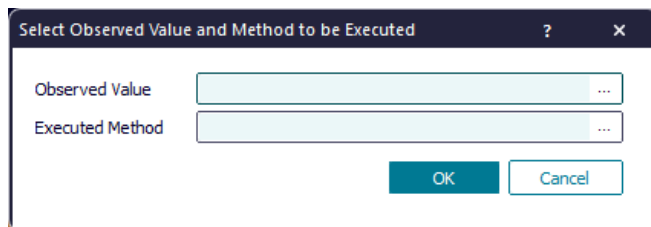
- Open the dialog of *Station1*. Select **Tools > Edit Observers**.



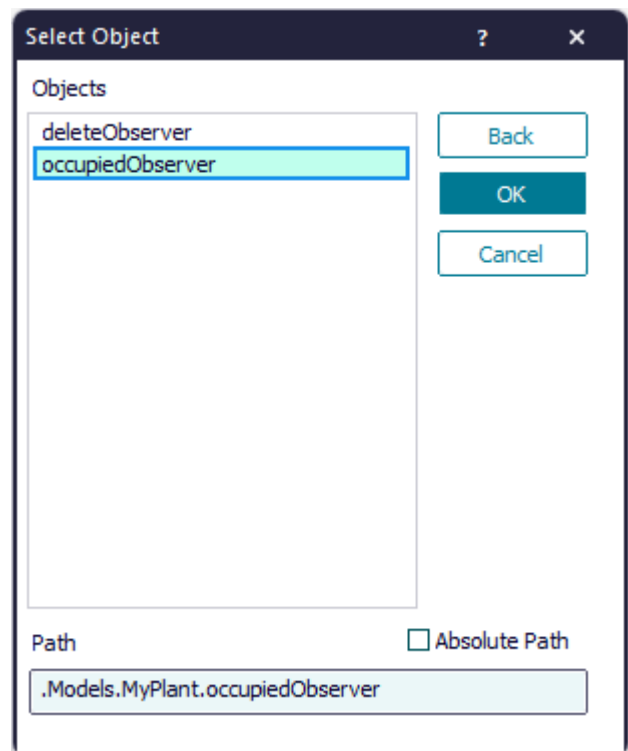
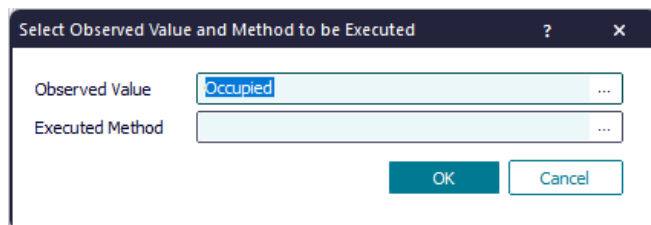
- To create the observer, click **New** in the dialog **Observers**.



- To select the *attribute/read-only attribute/method* whose value you want to watch, click ... next to **Attribute** in the dialog **Select Observed Value and Method to be Executed**. Then, select the attribute in the dialog **Select Object**. We selected **Occupied**.

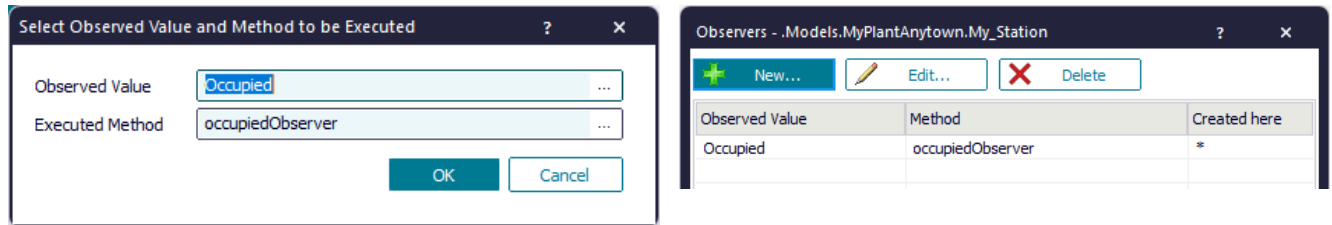


- To select the *Method* that will be executed, when the value of the *attribute/read-only attribute/method* changes, click ... next to **Method**. Then, select the *Method* in the dialog **Select Object**. We selected the *Method* named *occupiedObserver* in which we programmed our transfer behavior.



As the *Method* you can also use a *user-defined attribute* of data type *method*. To do so, right-click into the text box *Method* and select the command **Create Control**. In this case *Plant Simulation* automatically formats the *Method*.

- The dialog **Observers** then shows that our observer consists of the watched read-only attribute *Occupied* and of the method *occupiedObserver*. The asterisk (\*) shows that we created the observer in *Station1*, meaning that it is not inherited from its origin.



- The source code of our method *occupiedObserver* looks like this:

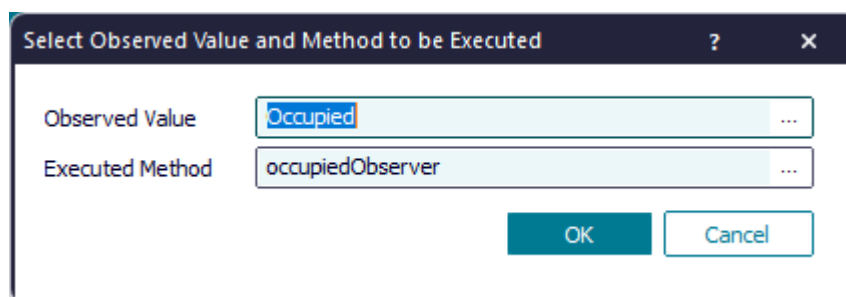
```
param attribute: string, oldValue: any
if ?.occupied
    ?.cont.move(Station2)
end
```

- First, we declare the two parameters, which are passed to the *Method*:
  - ◊ The name of the *attribute/read-only attribute/method* whose watchable value changed. This allows you to use a single *Method* as the method to be called for several attributes.
  - ◊ The previous value of the *attribute/read-only attribute/method*. This way you can still access the previous value after the executed method changed it to the new value.
- Then, we tell *Plant Simulation* to check if *Station1* is occupied, i.e., if it contains a part. If this is the case, we want it to move the part to *Station2*. Within the called *Method*, we can use the anonymous identifiers *?* and *@* to address the object, whose *read-only-attribute* changed, *Station1* in our case.
- To delete the observer, you can either click **Remove** in the dialog or program a method.

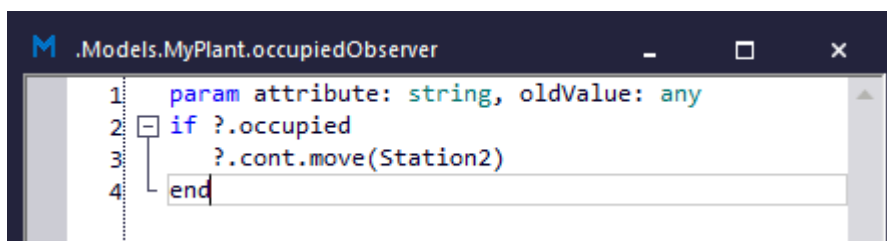
For our example above we entered the following source code:

```
Station1.removeObserver("Occupied","OccupiedObserver")
// "Occupied" is the name of the observer, "OccupiedObserver" is the
name of the method
```

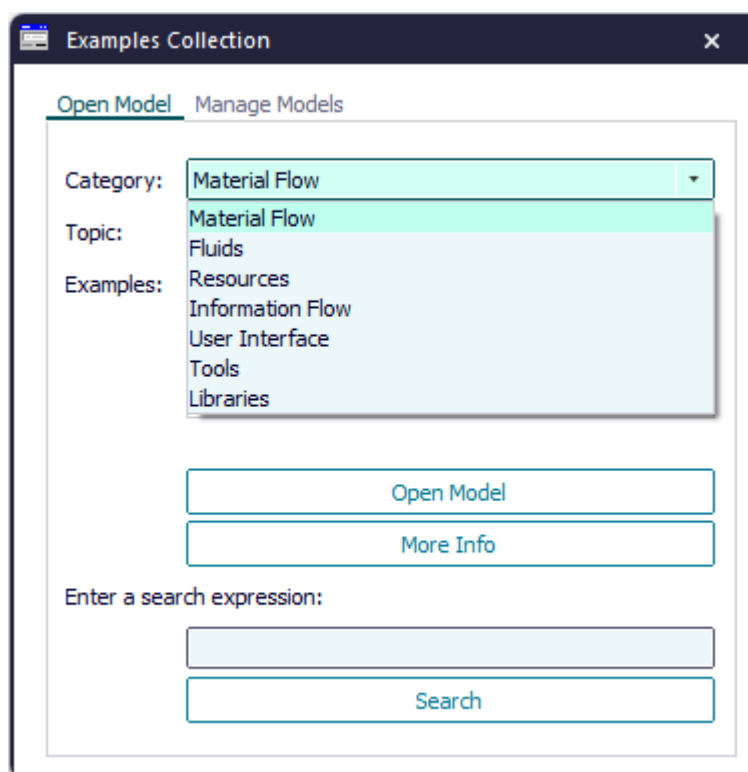
- To open the dialog for selecting the watched value, you can also double-click the text box **Observed Value** in the dialog **Observers**.



To open the window of the **Method**, which will be executed, you can also double-click the text box **Method**.



Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



Back to [Model the Flow of Materials, Advanced](#)



## Working with a Distance Control without AGVPool

### Work with a Distance Control without AGVPool

To prevent a *Transporter*, which reduces its speed, from being rear-ended by the following *Transporter*, the following *Transporter* has to reduce its speed in time as well.

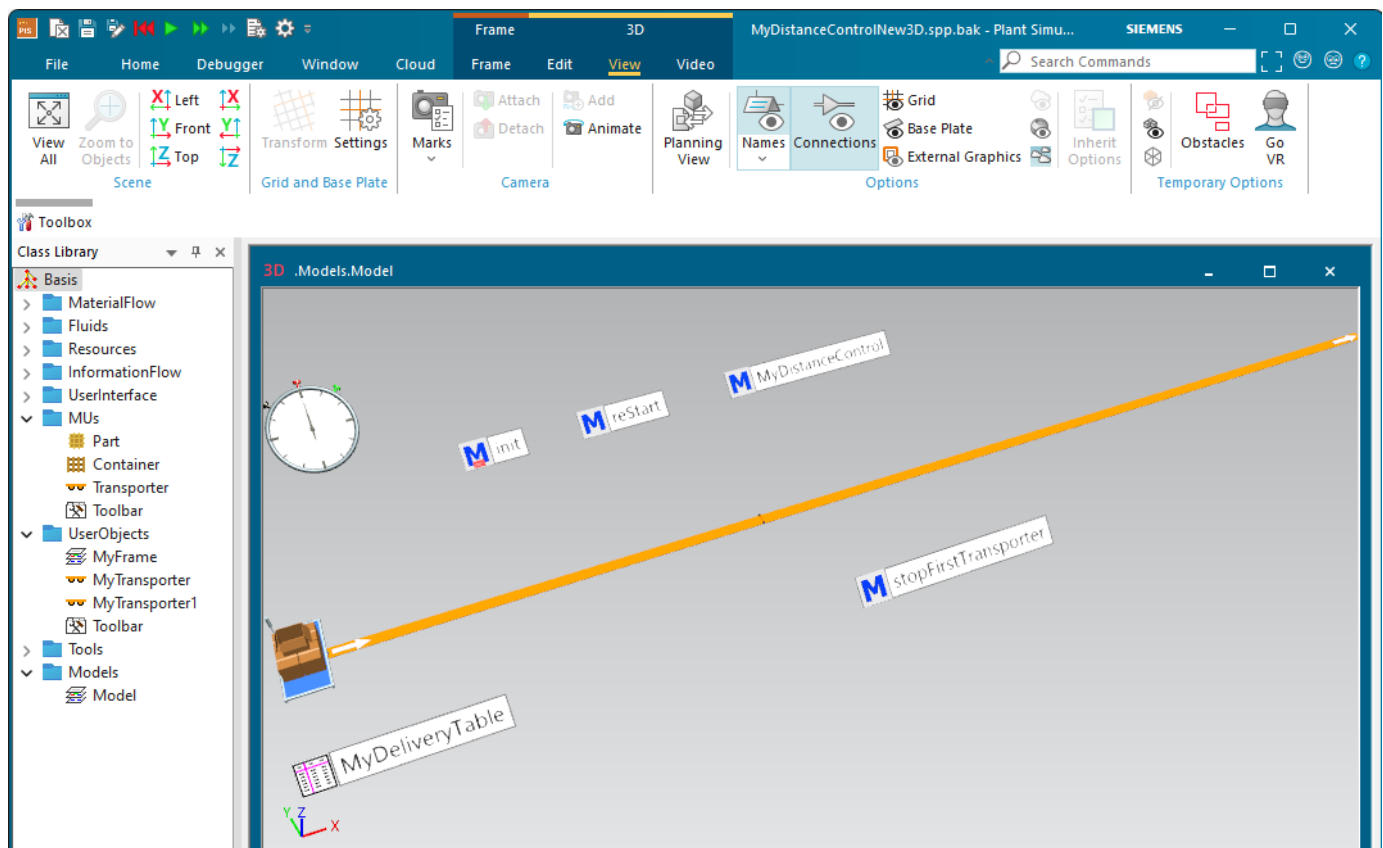
You can accomplish this in the **Distance Control** of the *Transporter*.

In our example the braking distance is 5 meters. The **Distance Control** is called at a **distance** of 6 meters to ensure a safety distance of 1 meter for any stopped *Transporter*.

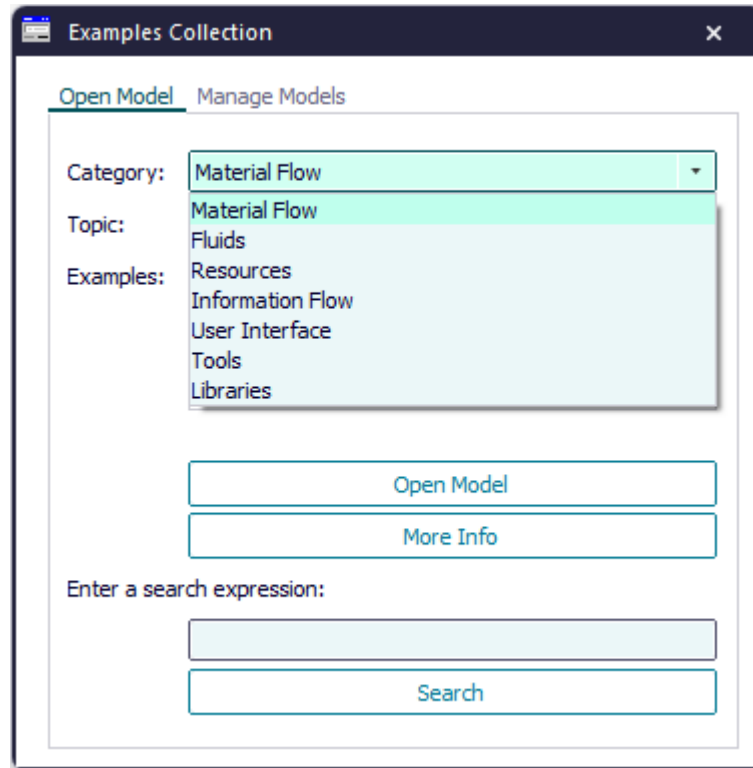
To create the sample model, we will:

- **Configure the Source That Produces the Transporters**
- **Configure the Track on Which the Transporters Drive**
- **Configure the Transporter That Drives on the Track**

The finished simulation model looks like this:



Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



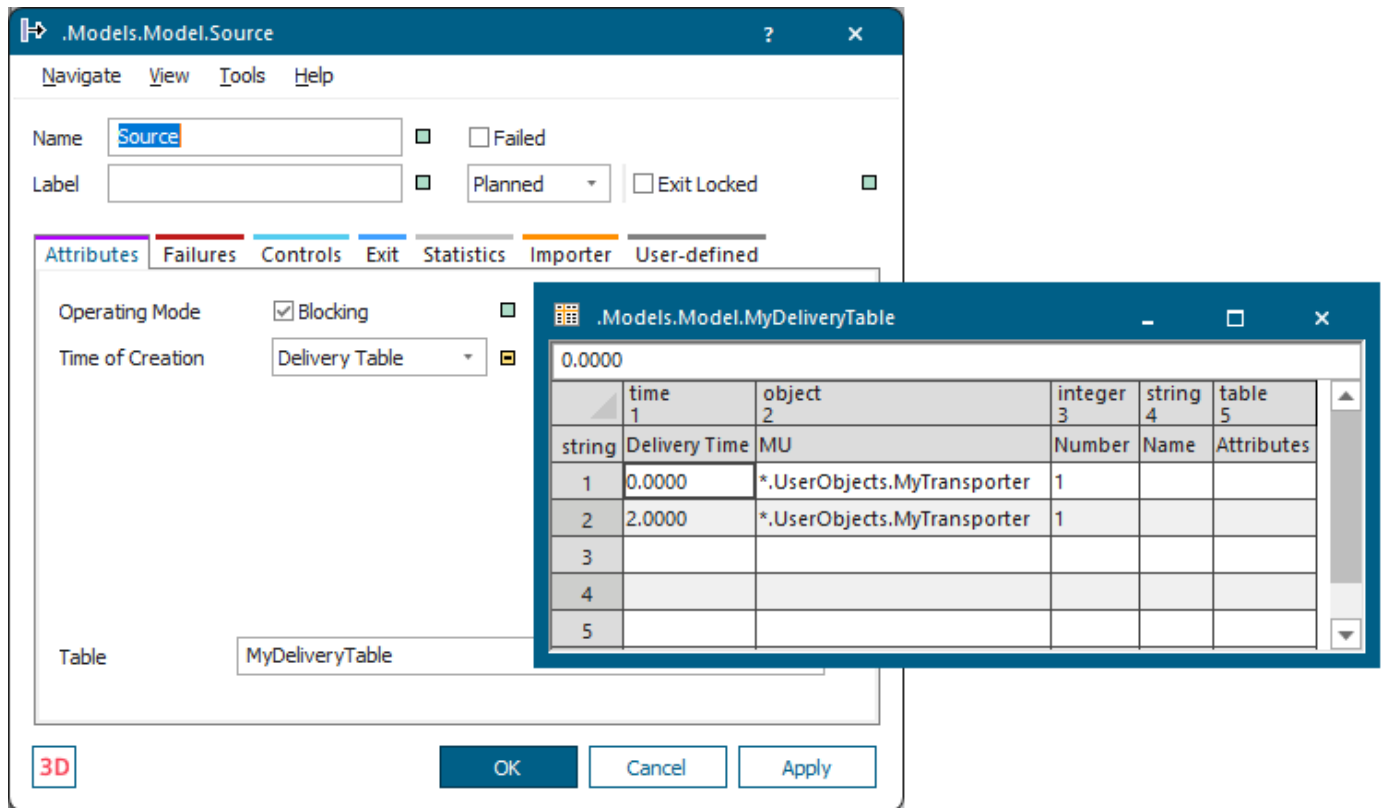
Compare [Prevent AGVs from Colliding](#)

Back to [Model the Flow of Materials, Advanced](#)

## Configure the Source That Produces the Transporters

The *Source* produces our vehicle, named **MyTransporter1**, which we duplicated within the folder **MUs** in the *Class Library* and which we renamed to **MyTransporter** in the folder **UserObjects**.

It creates them according to the settings which we entered into the **Delivery Table**, i.e., it produces the first *Transporter* at the time 0 minutes and the second of the two *Transporter* at 2 minutes into the simulation.



Back to [Work with a Distance Control without AGVPool](#)

## Configure the Track on Which the Transporters Drive

To make the first *Transporter* stop on the *Track* on which it drives, we added a *sensor* to the *Track* at the position 10 meters.

|                |                 |     |        |     |
|----------------|-----------------|-----|--------|-----|
| Distance       | self.OnDistance | ... | 1      | m   |
| Zone 1         | Length          | 1   | ΔWidth | 0.3 |
| Zone 2         | Length          | 2   | ΔWidth | 0.6 |
| Shift Calendar | ...             |     |        |     |

Sensors.ID1

Position

Length

10

[m]

☒ Front
 ☐ Rear
 ☐ Booking Point
 ☐ Light Barrier Mode

Activate

Always

Control

stopFirstTransporter

OK

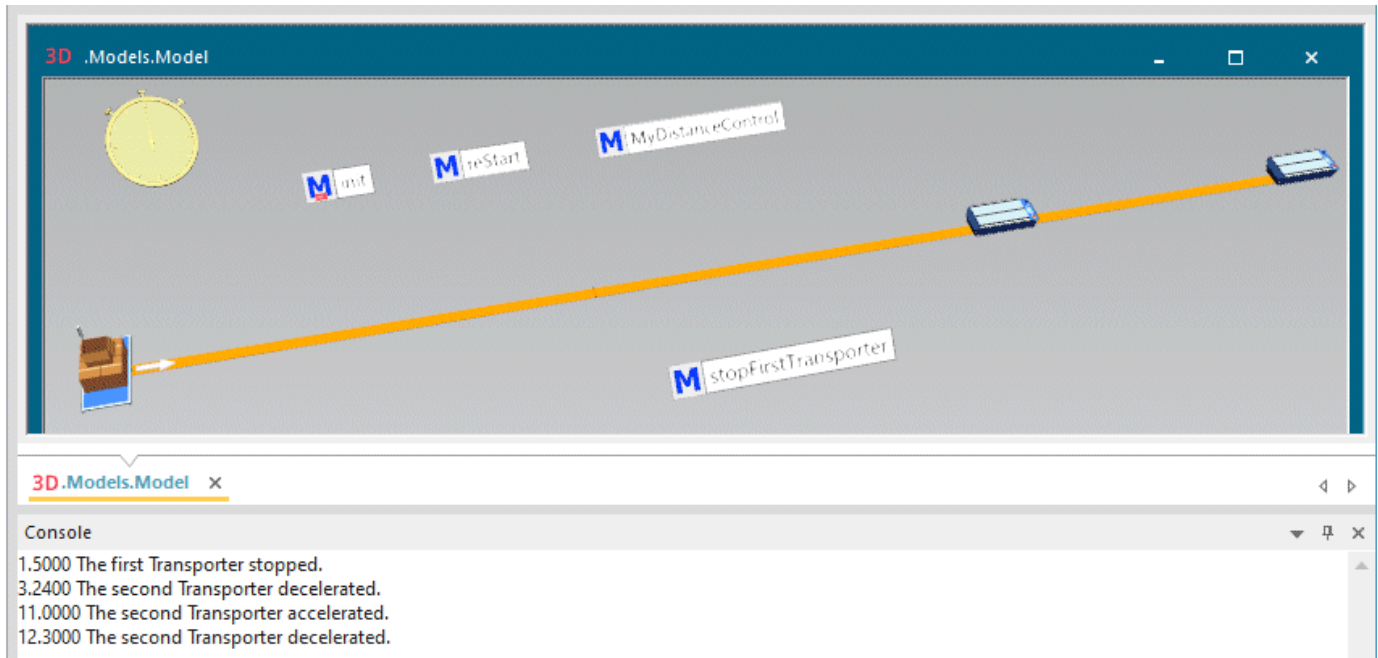
Cancel

Apply

The source code, which we entered into the **control** attached to the *sensor*, stops the first *Transporter* and shows a message with the time at which it stopped in the *Console*.

The source code looks like this:

```
param sensorID: integer, Front: boolean
if @.ID = 1
  @.Speed := 0
  print EventController.simTime, " The first Transporter stopped."
end
```



Back to [Work with a Distance Control without AGVPool](#)

## Configure the Transporter That Drives on the Track

To configure the *Transporter* with the settings you need, double-click its icon in the folder *MUs* in the *Class Library*.

As compared to the default settings of the built-in *Transporter* we activated **Acceleration** and entered an **Acceleration** and a **Deceleration** of  $10 \text{ m/s}^2$  each in our *Transporters* called *MyTransporter*.

**.UserObjects.MyTransporter**

Navigate View Tools Tabs Help

Name:  ☐ Stopped

Label:  ☐ Failed

Attributes Load Bay Routing Failures Controls Battery Product Statistic

**MU Size**

Length:  m

Width:  m

Height:  m

**Booking Point**

[0..1] (0.80 m)

[0..1] (0.40 m)

[0..1] (0.00 m)

Speed:  m/s

☒ Acceleration ☐ Backwards

Acceleration:  m/s²

Current Speed: 0 m/s

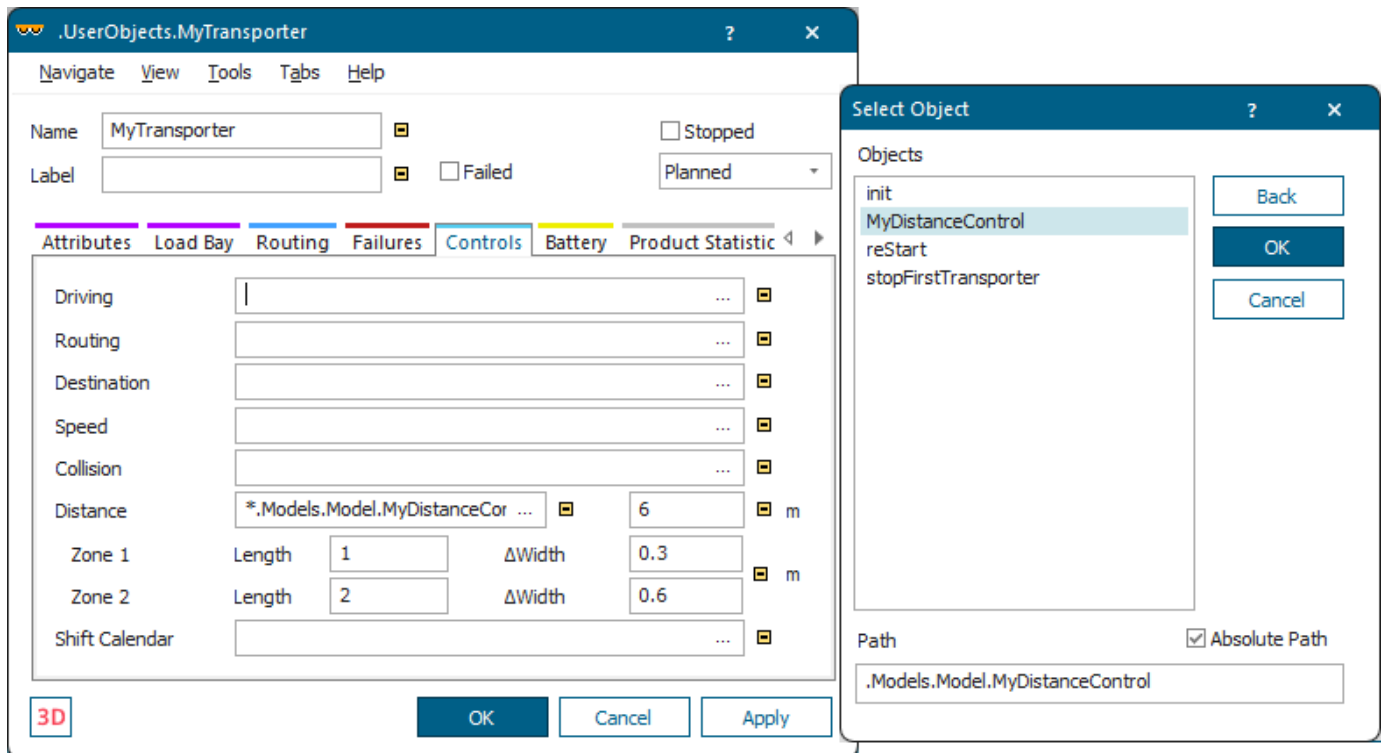
Deceleration:  m/s²

☐ Is Tractor

Start Delay Duration:

☒ 3D

Next, click the tab **Controls** and select the name of the **Distance Control** and enter the distance at which it will be called. We named our **Distance Control** *MyDistanceControl* and we entered a **Distance** of 6 meters.



### Note

When you select a control in the *class* of the object, make sure to activate **Absolute Path** in the dialog **Select Object**. Otherwise the *instances* will not find the selected control.

The **Distance Control** stops the second *Transporter* as soon as its distance to the first *Transporter*, which stopped, is 6 meters. It also shows a message with the time at which it stopped in the *Console*.

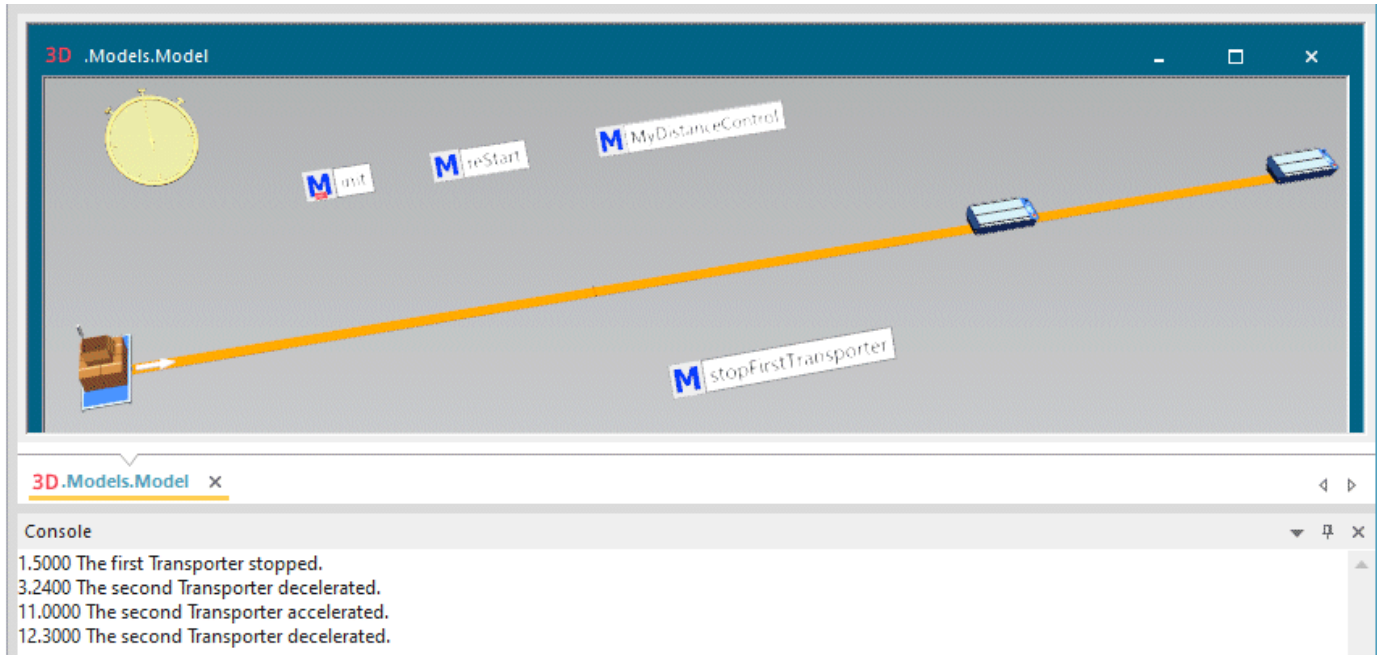
We entered the following source code in our **Distance Control** named *myDistanceControl*:

```
// This example only applies when acceleration is activated for the
// Transporter.
// For Transporters without acceleration you have to delay re-starting-up,
// for example with wait 0.01 before setting the speed. This prevents that
// the
// distance control will be called anew because the distance is exceeded
// again immediately.
// ? is the transporter for which you entered the distance control and the
// distance
// @ is the transporter for which the distance became too great or too small
param DistanceObjectBelowLimit: boolean
if DistanceObjectBelowLimit // once the distance between the transporters
?.Speed := 0 // is less than 6 meters, the following one
stops
print EventController.simTime," The second Transporter decelerated."
else
?.Speed := 10
```

```

    print EventController.simTime," The second Transporter accelerated."
    // when the distance reaches more than 6 meters, the second transporter
    starts again
end

```



To make the first *Transporter* continue its way on the *Track* after being stopped, we programmed an *init* method and entered the following source code:

```

// Restarts the first transporter after a certain time.
EventController.Speed := 60 // reduces the simulation speed to better see
                           // what is going on in the model
    &reStart.methCall(10)    // calls the method 'reStart' after 10 seconds

```

To set the **Speed** with which the first *Transporter* continues driving on the *Track*, we programmed the method *reStart*. It is called by the *init* method after 10 seconds and sets the speed of the first *Transporter* to 10 m/s<sup>2</sup>.

```
Track.MU(1).Speed := 10
```

Once the distance between the *Transporters* is more than 6 meters again, the second *Transporter* accelerates as well. This is because *Plant Simulation* calls the **Distance Control** when the **Distance** becomes too small **and** when it becomes too great.

Back to [Work with a Distance Control without AGVPool](#)